HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO THỰC HÀNH
DỊCH NGƯỢC
AN TOÀN GAME

Giảng viên hướng dẫn: Cao Minh Thắng
Lớp: D24CQGA02-B

Sinh viên **MSV**
Nguyễn Thành Vinh B24DCGA162

HÀ NỘI - 10/2026

MỤC LỤC

1. Môi trường và dữ liệu thực hành 2
   1.1. Gói mẫu và thư mục làm việc 2
   1.2 Khởi chạy UABEA và dnSpy 3
   1.3 Khởi chạy Il2CppDumper và Ghidra 4
   1.4 Mã nhận diện thư viện được phân tích 5
2. Khảo sát APK và xuất tài nguyên Unity 6
   2.1. Cấu trúc APK sau khi giải nén 6
   2.2. Xem trước tài nguyên hình ảnh 7
   2.3. Kết quả xuất asset 8
3. Tạo dữ liệu tra cứu IL2CPP 9
   Đầu ra và cách sử dụng dữ liệu dump 9
4. Tìm ứng viên bằng dnSpy và dump.cs 10
   4.1. Xác định field chứa số coin 10
   4.2. Danh sách phương thức ứng viên và đối chiếu RVA 11
5. Import và gắn tên phương thức trong Ghidra 12
   5.1. Import đúng thư viện ARM64 12
   5.2. Quy ước địa chỉ 13
   5.3 Khắc phục môi trường chạy script và hoàn tất gắn tên 13
6. Xác định nơi đọc và ghi field coin trong mã native 14
   6.1. Loại trừ Awake khỏi luồng nạp coin 15
   6.2. Logic coin khi vào game 15
   6.3. Đối chiếu khóa lưu và kết luận luồng khởi tạo 16
7. Logic cập nhật coin sau khi nhận thưởng 16
   7.1. Hàm StateCoin cộng trừ và lưu số dư 16
   7.2. Nhánh thưởng 100 coin trong luồng NextLevel 17
   7.3. Điều kiện bắt đầu nhận thưởng 18
   7.4. Đối chiếu tên cờ và nơi đặt lại 19
   7.5. Gắn callback cho các nút nhận thưởng 20
      BindClaimActions đăng ký NextLevel và ClaimX2Reward 21
   7.6. Sự kiện nhả chuột thực thi callback của UIClick 23
   7.7. Điều kiện thắng và mở giao diện nhận thưởng 24
   7.8. Tổng hợp luồng và giới hạn chứng minh 27
8. Nhận xét an toàn và giới hạn phân tích tĩnh 28
   8.1. Kịch bản can thiệp tài nguyên phía client 28
   8.2 Tác động và giới hạn của kết luận 28
   8.3 Cách giảm phụ thuộc vào dữ liệu do client quyết định 28
9. Kết luận 29

NỘI DUNG BÁO CÁO

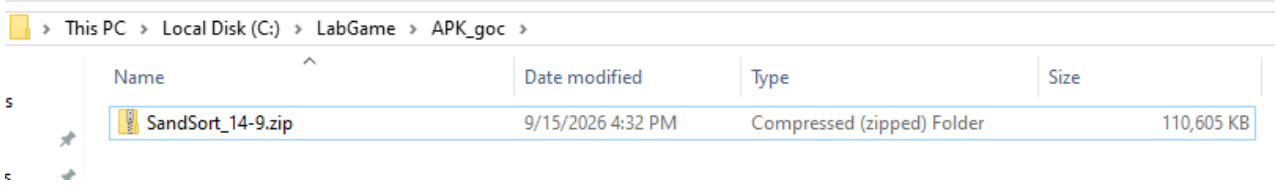
1. Môi trường và dữ liệu thực hành

Môi trường thực hành là Windows trong máy ảo VMware Workstation có tên thuchanhdichnguoc1. Dữ liệu và công cụ được đặt trong C:\LabGame. File mẫu SandSort_14-9.zip nằm trong APK_goc, còn dữ liệu đã giải nén nằm trong APK_giai_nen. Thư viện được chọn để phân tích là bản ARM64.

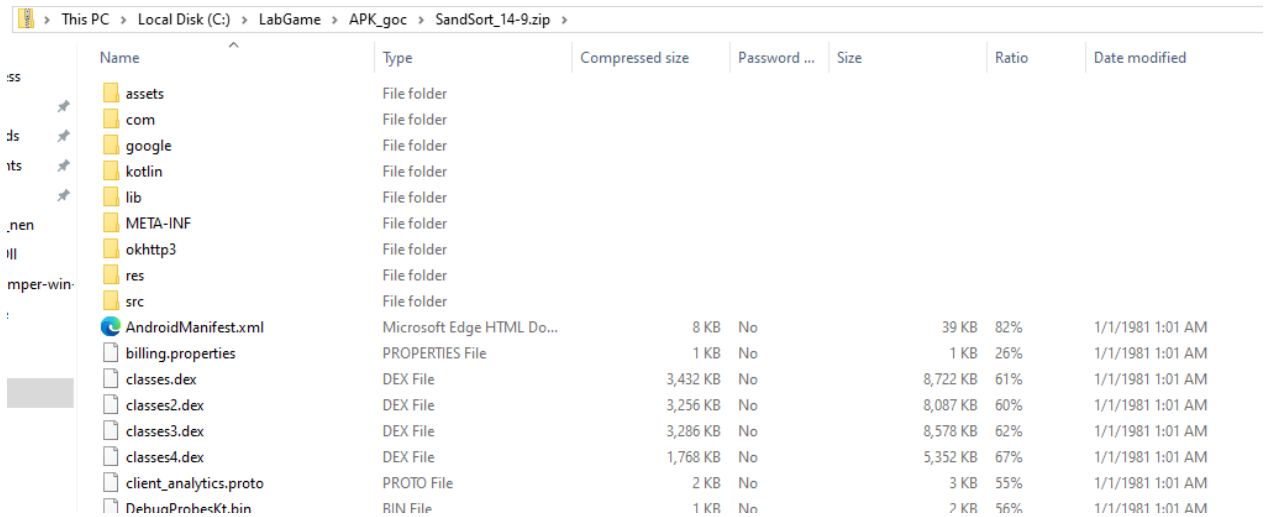
Thành phần	Thông tin và trạng thái
Gói mẫu game	SandSort_14-9.zip, lưu tại C:\LabGame\APK_goc. Tên và đuôi file được xác nhận bằng ảnh File Explorer.
Thư viện được chọn	C:\LabGame\APK_giai_nen\lib\arm64-v8a\libil2cpp.so; ELF AARCH64, little-endian, địa chỉ 64 bit.
UABEA	Gói uabea-windows; chạy UABEANext4.Desktop.exe thành công. Đã xem Preview và xuất Texture2D ở phần thực hành.
Il2CppDumper	Gói v6.7.46; ứng dụng khởi chạy đến hộp chọn file đầu vào. Lần phân tích trước đã tạo dump.cs, DummyDll và script.json.
dnSpy	v6.1.8, 64 bit .NET; cửa sổ Assembly Explorer mở được. Đã tra cứu DummyDll của game.
Ghidra	Gói v12.1.2; ghidraRun.bat mở được cửa sổ quản lý project. Đã phân tích thư viện và chạy script gắn tên thành công.

1.1. Gói mẫu và thư mục làm việc

File mẫu được lưu tại C:\LabGame\APK_goc\SandSort_14-9.zip, có dung lượng hiển thị là 110 605 KB. Khi mở file, các thư mục assets, lib, res cùng AndroidManifest.xml và các file classes.dex xuất hiện ở thư mục gốc. Cấu trúc này phù hợp với một gói ứng dụng Android. Tuy nhiên, ảnh hiện có chỉ xác nhận tên file ZIP, chưa xác định tên APK ban đầu hoặc phiên bản game.

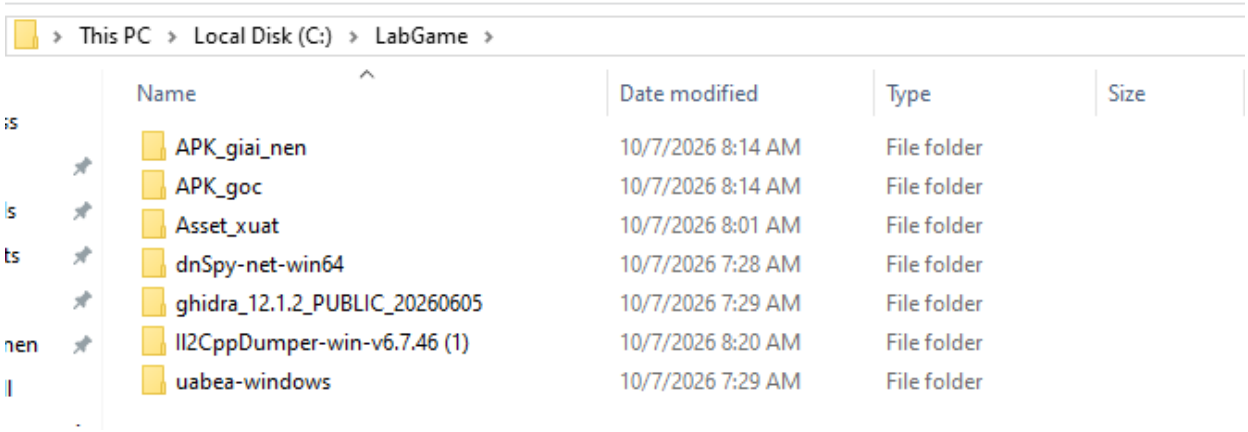


Hình 1. Tên gói mẫu SandSort_14-9.zip trong thư mục APK_goc



Hình 2. Cấu trúc Android hiện trực tiếp trong gói SandSort_14-9.zip

Trong C:\LabGame, APK_goc chứa file mẫu, APK_giai_nen chứa dữ liệu đã giải nén và Asset_xuat chứa tài nguyên đã xuất. Các thư mục còn lại chứa công cụ phân tích. Kết quả từ Il2CppDumper được lưu trong Il2CppDumper-win-v6.7.46 (1).



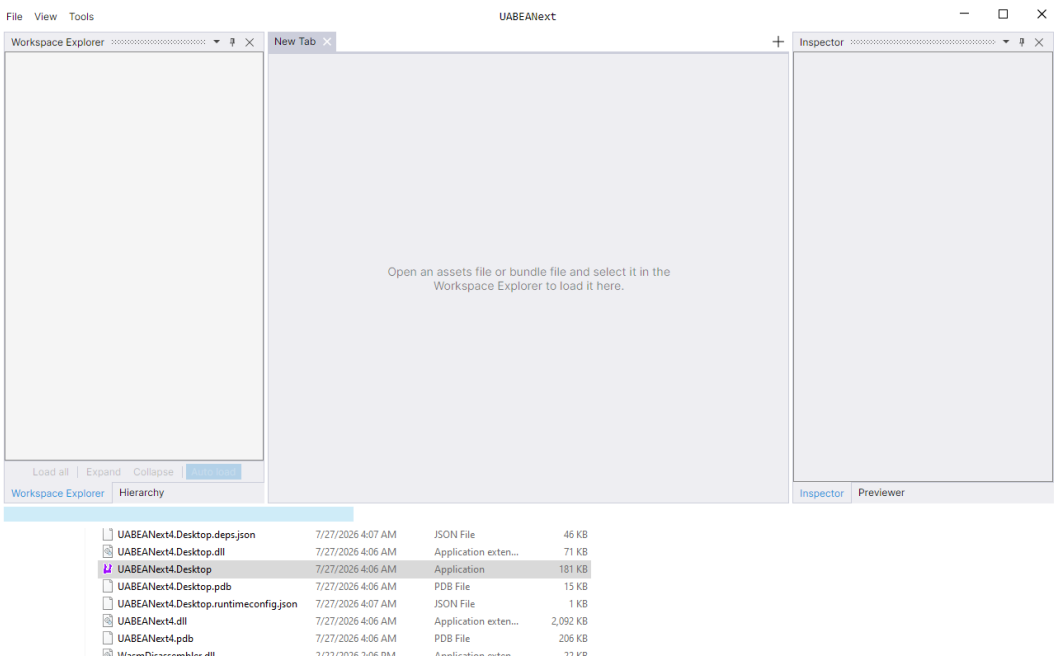
Hình 3. Thư mục làm việc LabGame với dữ liệu và các công cụ đã giải nén

Project Ghidra được đặt tên là dichnguoc. Các file dump.cs, script.json và thư mục DummyDll được lưu trong C:\LabGame\Il2CppDumper-win-v6.7.46 (1) để tra cứu và gắn tên phương thức.

1.2 Khởi chạy UABEA và dnSpy

UABEA

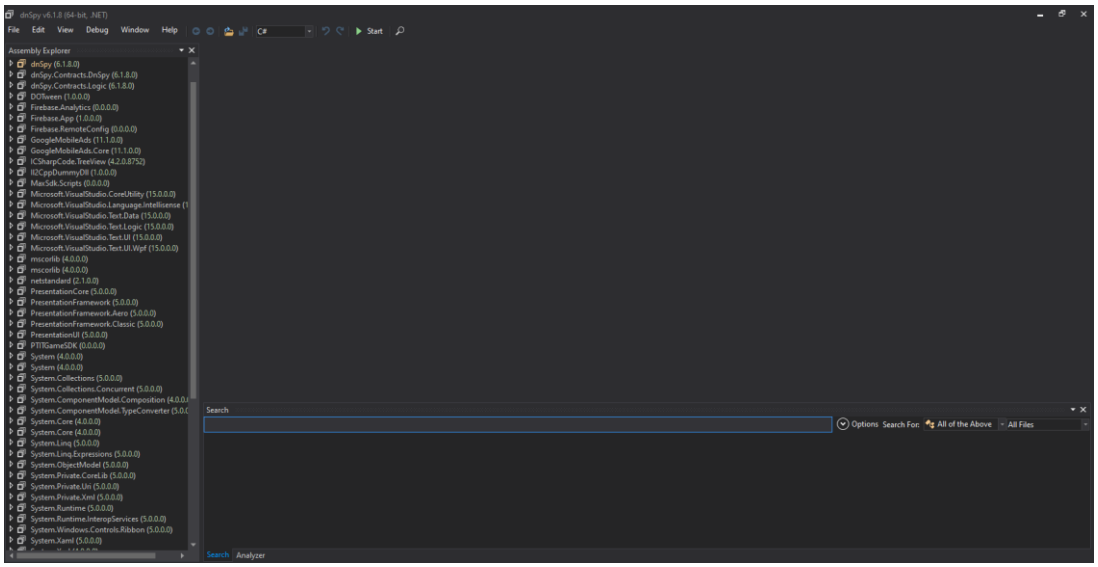
UABEANext4.Desktop.exe được mở từ C:\LabGame\uabea-windows. Cửa sổ hiển thị Workspace Explorer và Inspector. Công cụ này được dùng để xem và xuất asset ở phần 2.



Hình 4. UABEANext mở thành công và file chạy hiện trong thư mục công cụ

dnSpy

dnSpy.exe được mở từ C:\LabGame\dnSpy-net-win64. Thanh tiêu đề hiển thị phiên bản 6.1.8, 64 bit .NET. Assembly Explorer được dùng để tra cứu các lớp trong DummyDll ở phần 4.

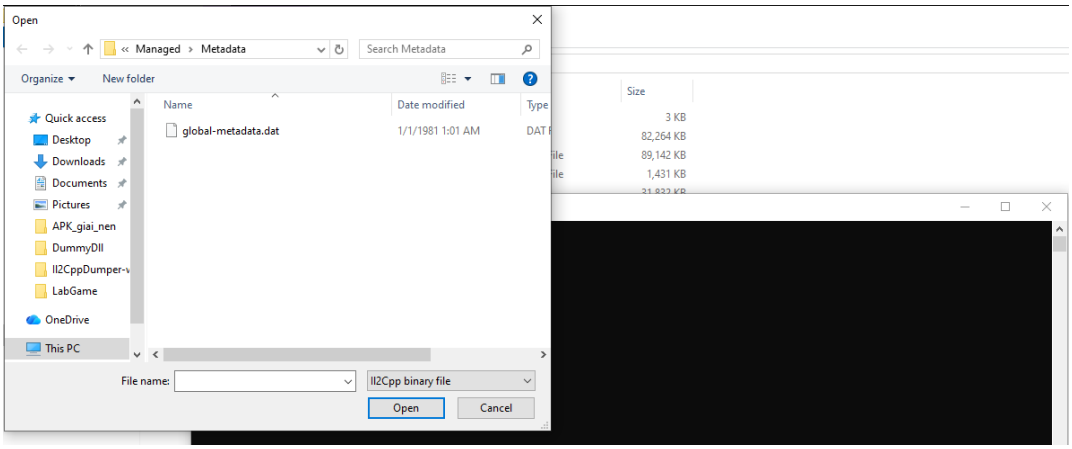


Hình 5. dnSpy v6.1.8 khởi chạy với bảng Assembly Explorer

1.3 Khởi chạy Il2CppDumper và Ghidra

Il2CppDumper

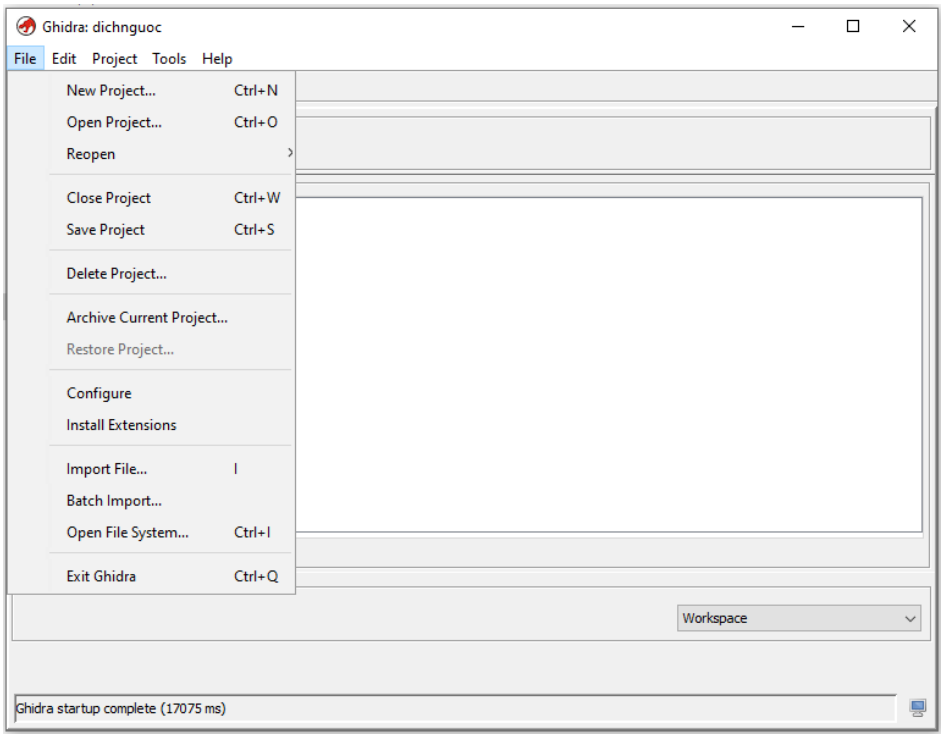
Il2CppDumper.exe được mở từ C:\LabGame\Il2CppDumper-win-v6.7.46 (1). Chương trình xuất hiện hộp chọn file binary như Hình 6. Ở bước này cần chọn libil2cpp.so trước, sau đó chọn global-metadata.dat trong hộp tiếp theo.



Hình 6. Il2CppDumper đã khởi chạy và chờ chọn file binary đầu vào

Ghidra

Ghidra được mở bằng ghidraRun.bat trong C:\LabGame\ghidra_12.1.2_PUBLIC_20260605\ghidra_12.1.2_PUBLIC. Cửa sổ quản lý hiển thị project dichnguoc và dòng Ghidra startup complete. Các bước import và phân tích thư viện được trình bày ở phần 5.



Hình 7. Ghidra hoàn tất khởi động và mở cửa sổ quản lý project

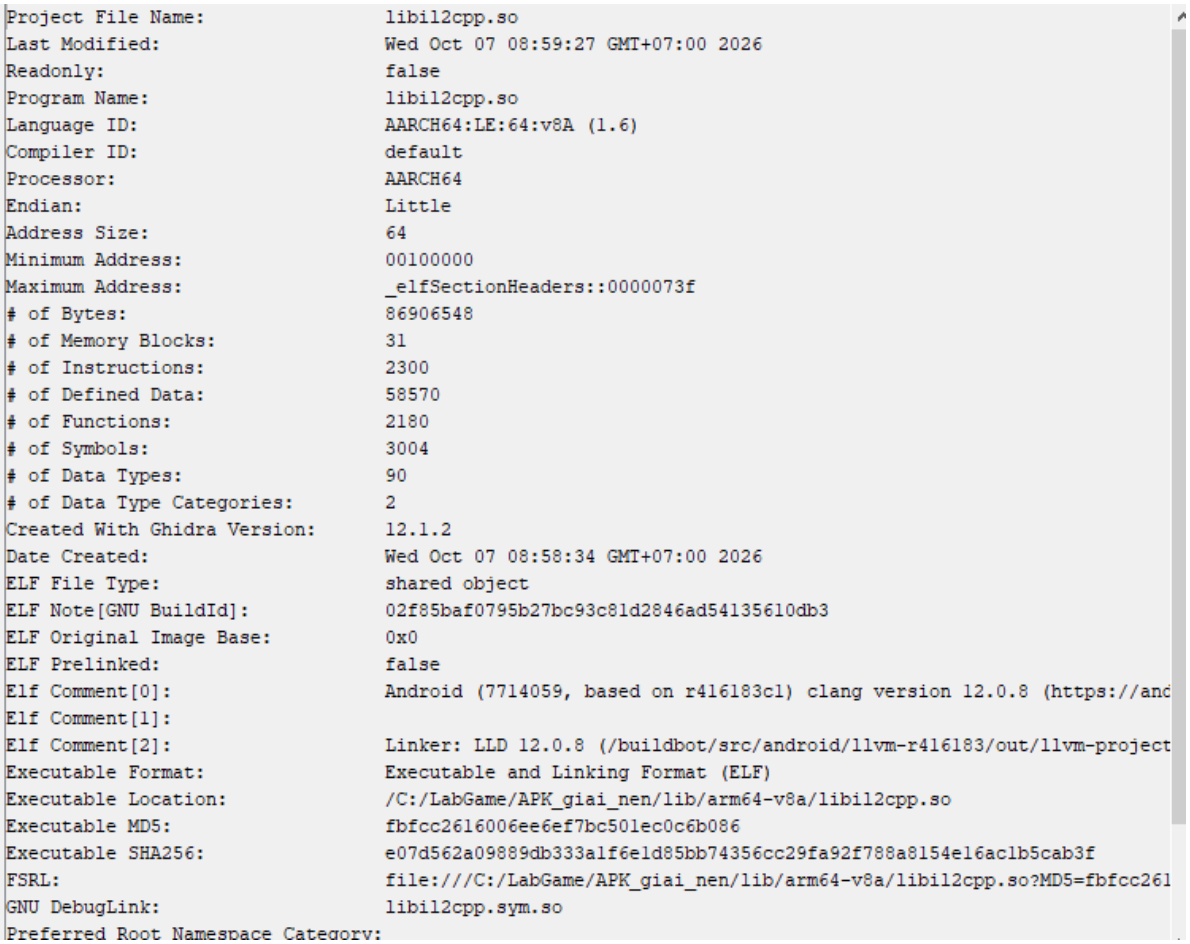
1.4 Mã nhận diện thư viện được phân tích

Thư viện được phân tích là C:\LabGame\APK_giai_nen\lib\arm64-v8a\libil2cpp.so. Import Results Summary của Ghidra hiển thị định dạng ELF shared object, processor AARCH64 và các mã nhận diện bên dưới. Các mã băm này thuộc libil2cpp.so, không phải toàn bộ gói APK.

GNU Build ID
02f85baf0795b27bc93c81d2846ad54135610db3

MD5 của libil2cpp.so
fbfcc2616006ee6ef7bc501ec0c6b086

SHA256 của libil2cpp.so
e07d562a09889db333a1f6e1d85bb74356cc29fa92f788a8154e16ac1b5cab3f



Project File Name:	libil2cpp.so
Last Modified:	Wed Oct 07 08:59:27 GMT+07:00 2026
Readonly:	false
Program Name:	libil2cpp.so
Language ID:	AARCH64:LE:64:v8A (1.6)
Compiler ID:	default
Processor:	AARCH64
Endian:	Little
Address Size:	64
Minimum Address:	00100000
Maximum Address:	_elfSectionHeaders::0000073f
# of Bytes:	86906548
# of Memory Blocks:	31
# of Instructions:	2300
# of Defined Data:	58570
# of Functions:	2180
# of Symbols:	3004
# of Data Types:	90
# of Data Type Categories:	2
Created With Ghidra Version:	12.1.2
Date Created:	Wed Oct 07 08:58:34 GMT+07:00 2026
ELF File Type:	shared object
ELF Note[GNU BuildId]:	02f85baf0795b27bc93c81d2846ad54135610db3
ELF Original Image Base:	0x0
ELF Prelinked:	false
Elf Comment[0]:	Android (7714059, based on r416183c1) clang version 12.0.8 (https://and
Elf Comment[1]:	
Elf Comment[2]:	Linker: LLD 12.0.8 (/buildbot/src/android/llvm-r416183/out/llvm-project
Executable Format:	Executable and Linking Format (ELF)
Executable Location:	/C:/LabGame/APK_giai_nen/lib/arm64-v8a/libil2cpp.so
Executable MD5:	fbfcc2616006ee6ef7bc501ec0c6b086
Executable SHA256:	e07d562a09889db333a1f6e1d85bb74356cc29fa92f788a8154e16ac1b5cab3f
FSRL:	file:///C:/LabGame/APK_giai_nen/lib/arm64-v8a/libil2cpp.so?MD5=fbfcc261
GNU DebugLink:	libil2cpp.sym.so
Preferred Root Namespace Category:	

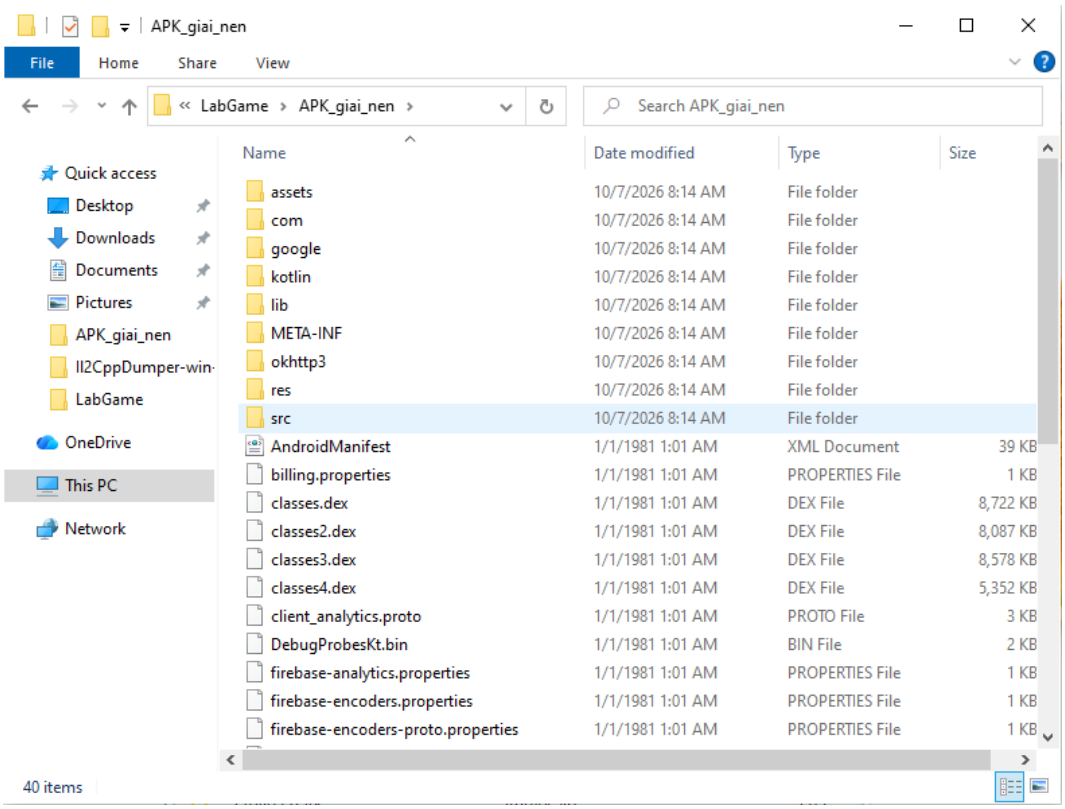
Hình 8. Thông tin kiến trúc và mã nhận diện của libil2cpp.so trong Ghidra

2. Khảo sát APK và xuất tài nguyên Unity

2.1. Cấu trúc APK sau khi giải nén

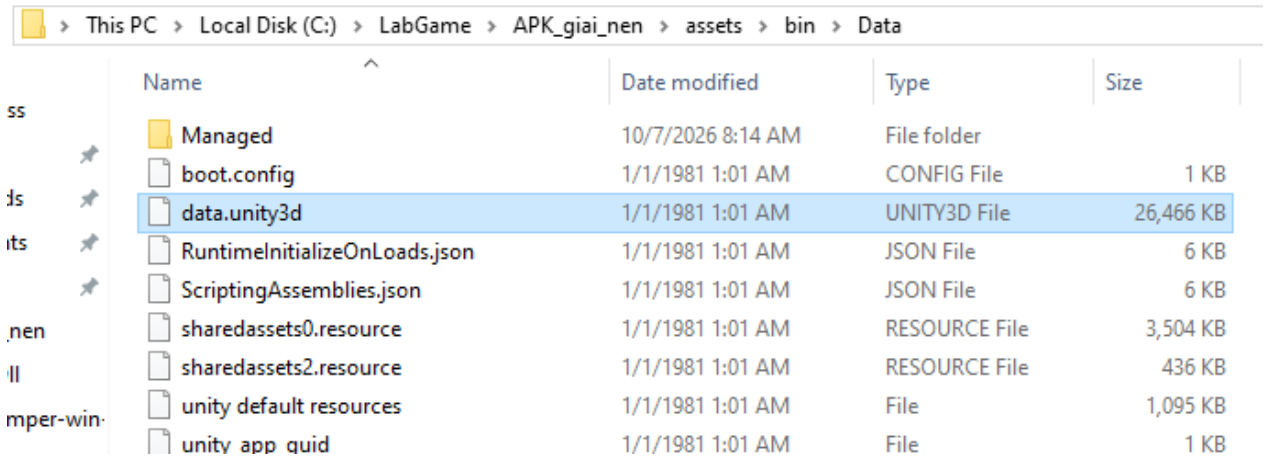
Dữ liệu được giải nén vào C:\LabGame\APK_giai_nen. Thư mục này có assets, lib, res, các file classes.dex, classes2.dex và AndroidManifest. assets chứa dữ liệu Unity, còn lib chứa thư viện native theo kiến trúc. Các file DEX và manifest thuộc phần ứng dụng Android; mã IL2CPP cần phân tích nằm trong thư viện native.

File được mở bằng UABEA là C:\LabGame\APK_giai_nen\assets\bin\Data\data.unity3d. Trong UABEA, sharedassets0.assets, sharedassets1.assets và sharedassets2.assets được hiển thị bên trong workspace data.unity3d. sharedassets0.assets được chọn để tìm tài nguyên hình ảnh.



Hình 9. Cấu trúc thư mục APK_giai_nen sau khi giải nén

Hình 10 cho thấy vị trí file data.unity3d trong thư mục đã giải nén. Nội dung của file được mở bằng UABEA ở mục tiếp theo.

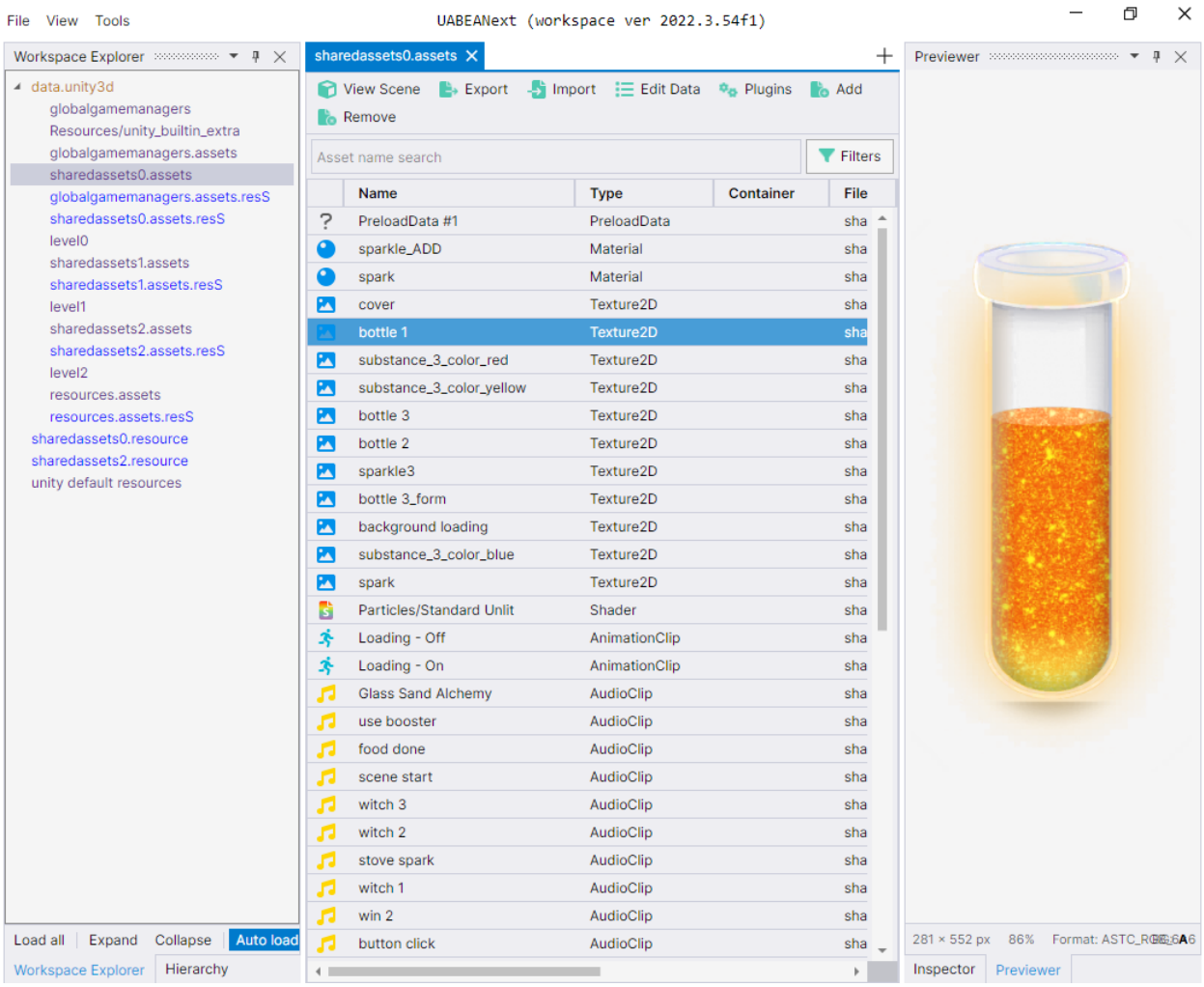


Hình 10. Đường dẫn data.unity3d trong thư mục APK_giai_nen

2.2. Xem trước tài nguyên hình ảnh

Trong workspace data.unity3d, asset bottle 1 được tìm thấy ở mục sharedassets0.assets và có loại Texture2D. Preview hiển thị một chai chứa vật liệu màu vàng cam. Kích thước texture là 281 × 552 pixel.

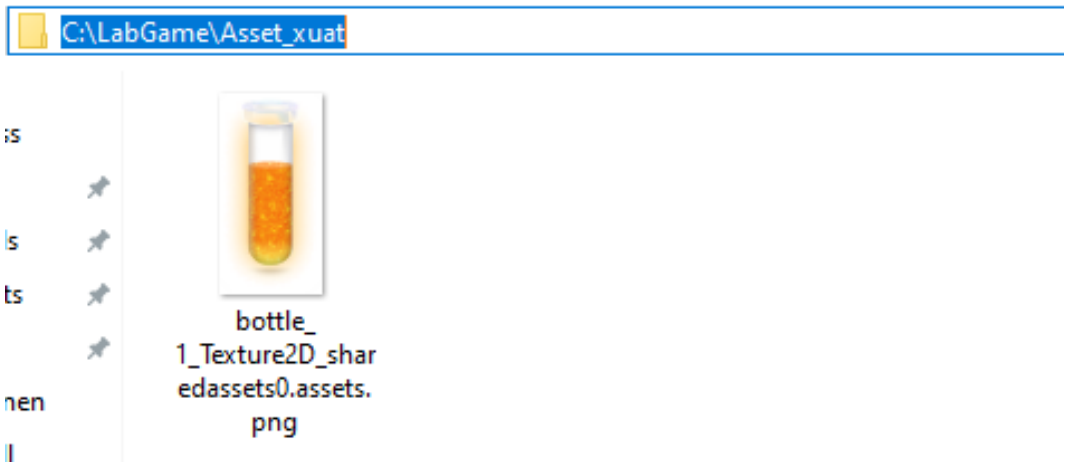
Hình ảnh trong Preview cho thấy texture này có thể được dùng để hiển thị chai chứa cát hoặc vật liệu màu trong game. Tuy nhiên, prefab và scene chưa được kiểm tra để xác định chính xác vị trí sử dụng.



Hình 11. Asset bottle 1 thuộc loại Texture2D trong sharedassets0.assets và ảnh Preview

2.3. Kết quả xuất asset

Sau khi xem Preview, texture được xuất bằng chức năng Export của UABEA vào C:\LabGame\Asset_xuat. File xuất có tên bottle_1_Texture2D_sharedassets0.assets.png và đuôi .png. Hình thu nhỏ của file giống asset bottle 1 trong Preview.

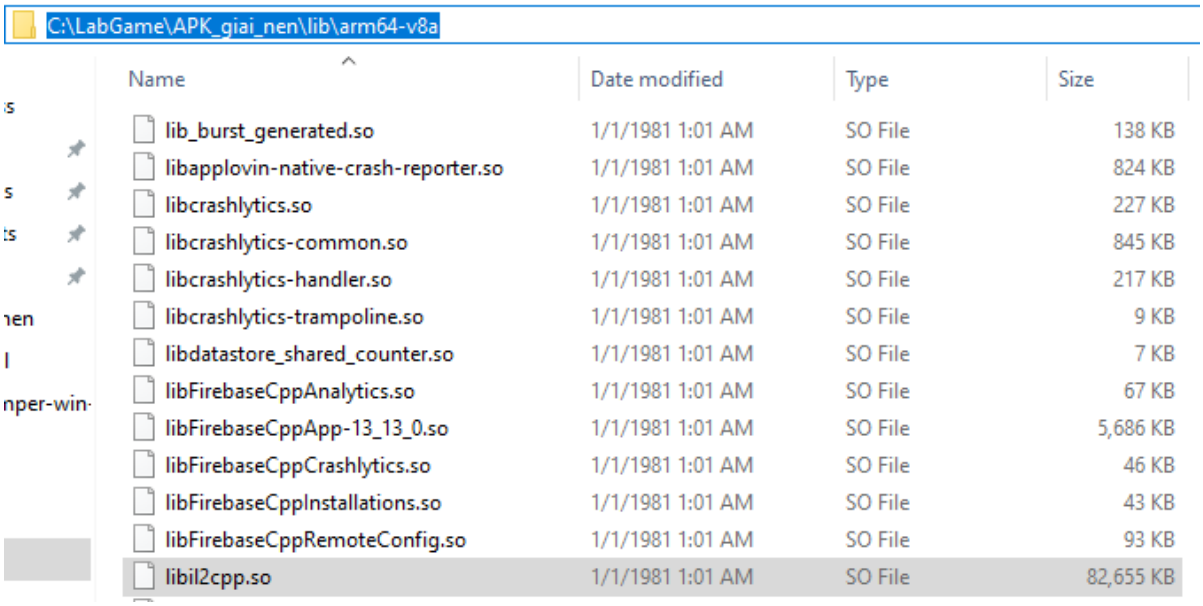


Hình 12. File bottle_1_Texture2D_sharedassets0.assets.png trong C:\LabGame\Asset_xuat

3. Tạo dữ liệu tra cứu IL2CPP

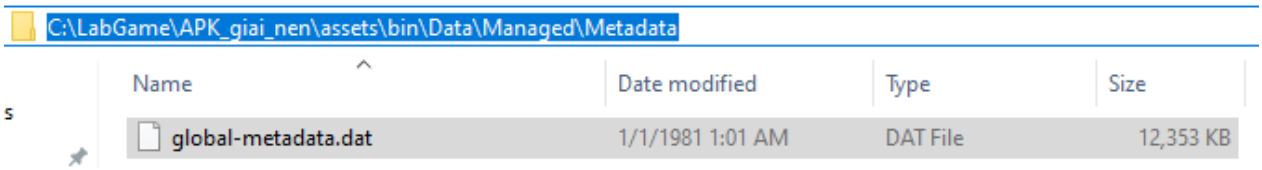
Il2CppDumper kết hợp libil2cpp.so với global-metadata.dat để tạo dữ liệu tra cứu lớp, phương thức và địa chỉ mã máy. Hai file lấy từ cùng gói đã giải nén. Bản libil2cpp.so ARM64 cũng được dùng khi import vào Ghidra.

Đầu vào binary: C:\LabGame\APK_giai_nen\lib\arm64-v8a\libil2cpp.so. Đây là bản ARM64 được dùng để phân tích trong Ghidra.



Hình 13. Vị trí libil2cpp.so ARM64 dùng làm đầu vào binary

Đầu vào metadata: C:\LabGame\APK_giai_nen\assets\bin\Data\Managed\Metadata\global-metadata.dat.



Hình 14. Vị trí global-metadata.dat dùng làm đầu vào metadata

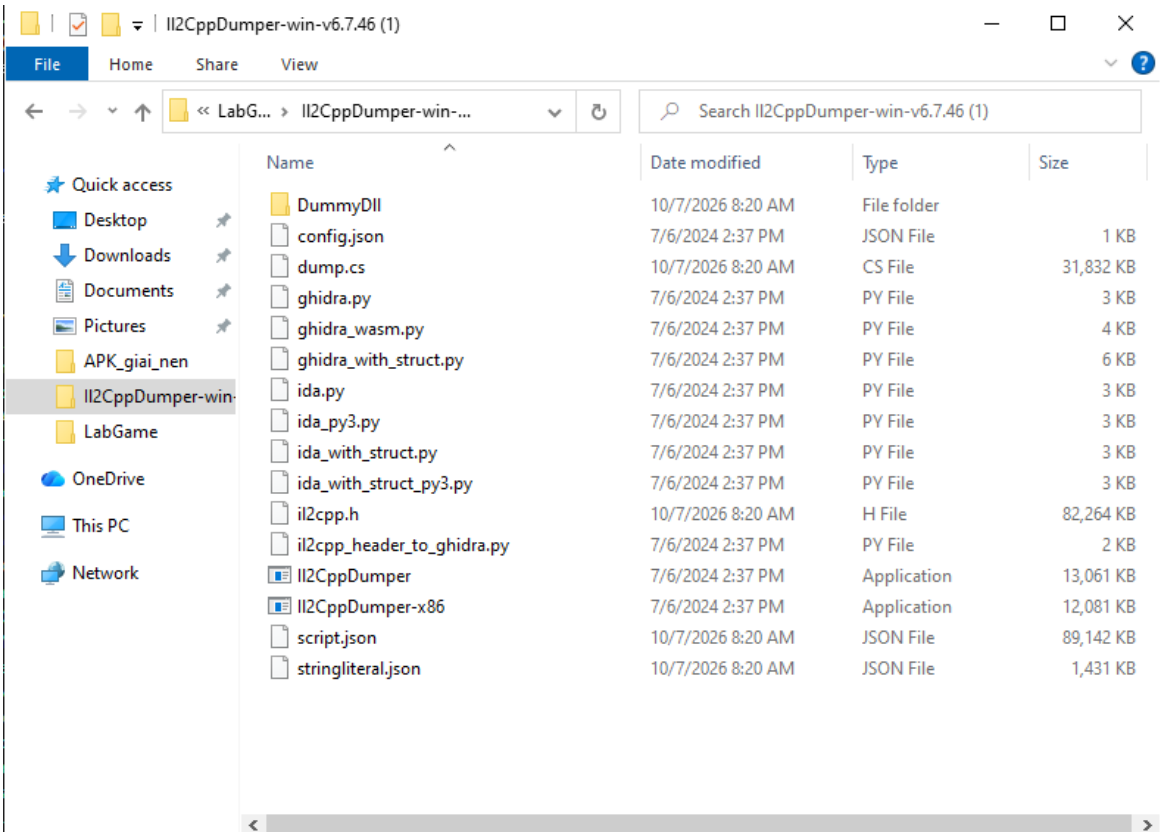
Hình 13 và Hình 14 cho thấy vị trí hai file đầu vào. Khi chạy Il2CppDumper, libil2cpp.so được chọn trước, sau đó là global-metadata.dat. Hai ảnh này được chụp trong File Explorer, không phải trong hộp chọn file của lần chạy dump.

Đầu ra và cách sử dụng dữ liệu dump

Thư mục Il2CppDumper-win-v6.7.46 (1) có DummyDll, dump.cs, script.json, ghidra.py, il2cpp.h và stringliteral.json. Các file này được dùng để tra cứu tên, cấu trúc và địa chỉ; chúng không phải toàn bộ mã nguồn C# gốc của game.

Đầu ra	**Vai trò trong bài**
dump.cs	Tra tên lớp, field, chữ ký phương thức, RVA và File Offset.
DummyDll	Mở Assembly-CSharp.dll trong dnSpy để xem cấu trúc namespace và thành viên.

Đầu ra	Vai trò trong bài
script.json	Cung cấp tên và địa chỉ để script gắn nhãn trong Ghidra.
ghidra.py	Áp dụng dữ liệu script.json vào chương trình native đã import.
il2cpp.h và stringliteral.json	Dữ liệu hỗ trợ về kiểu và chuỗi; chỉ sử dụng khi cần đối chiếu thêm.



Hình 15. Các file đầu ra của Il2CppDumper trong thư mục làm việc

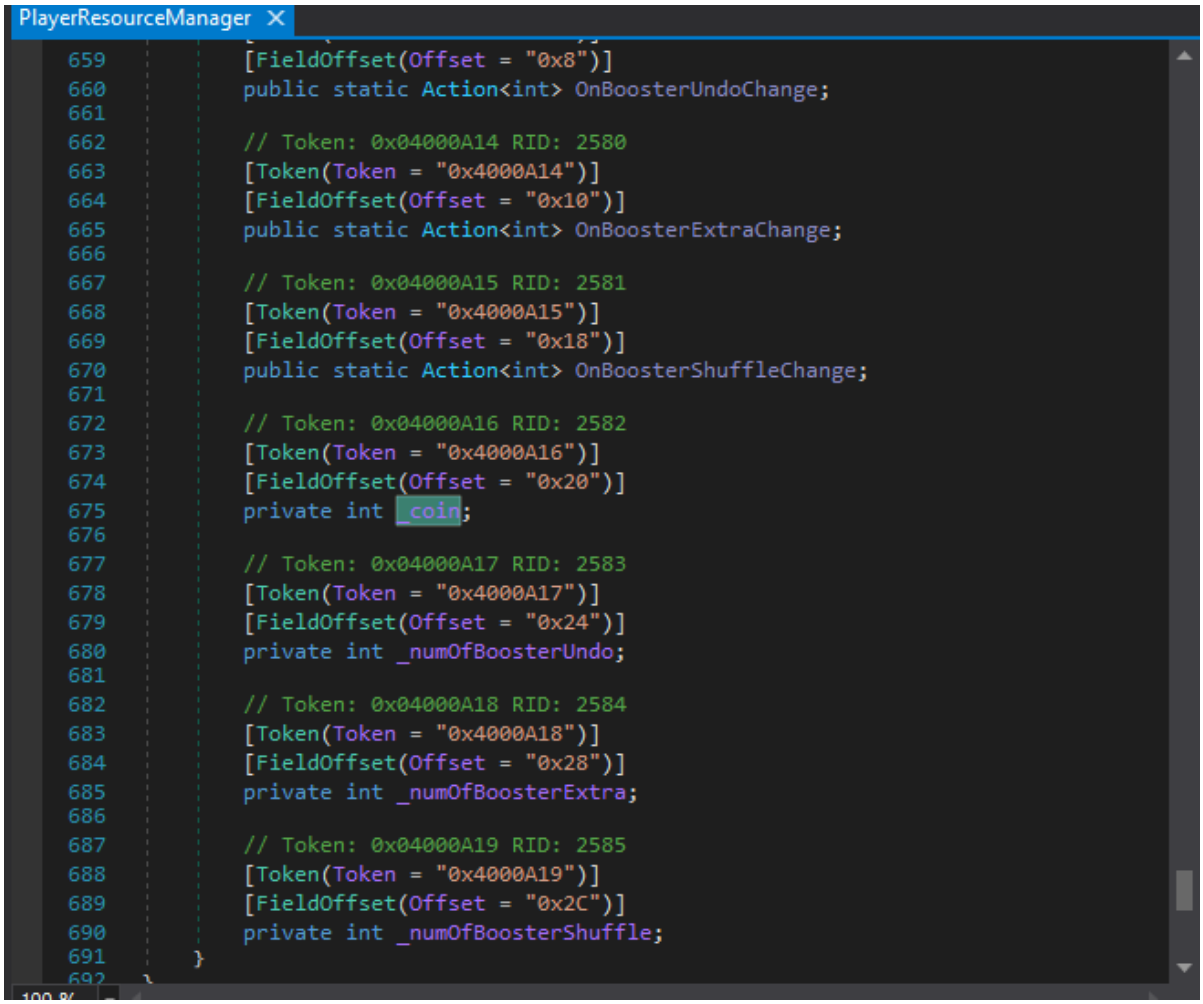
Tên lớp và phương thức được tìm trong DummyDll bằng dnSpy, sau đó tra phương thức tương ứng trong dump.cs để lấy RVA và field offset. Tiếp theo, ghidra.py đọc script.json để gắn tên trong Ghidra. Từ tên đã gắn hoặc địa chỉ tính bằng RVA + Image Base, có thể mở Listing và Decompiler để đọc mã native. DummyDll chỉ cung cấp thông tin phục vụ tra cứu, không chứa phần thực thi native của phương thức.

4. Tìm ứng viên bằng dnSpy và dump.cs

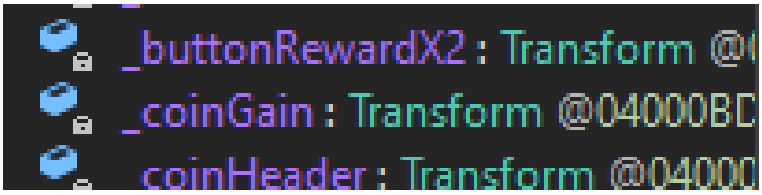
4.1. Xác định field chứa số coin

Khi mở DummyDll\Assembly-CSharp.dll trong dnSpy và tìm từ khóa coin ở namespace FoodSort, PlayerResourceManager có field private int _coin với FieldOffset 0x20. Kiểu int phù hợp với số lượng coin. Các lệnh đọc và ghi field này được kiểm tra tiếp trong mã native ở phần 6.

Kết quả tìm kiếm còn có _coinGain trong UIWinGame. Tuy nhiên, field này có kiểu Transform nên được loại khỏi ứng viên chứa số dư coin. Field int _coin của PlayerResourceManager được giữ lại để kiểm tra.



Hình 16. Field _coin kiểu int trong FoodSort.PlayerResourceManager có offset 0x20



Hình 17. Field _coinGain của UIWinGame có kiểu Transform, thuộc thành phần giao diện

4.2. Danh sách phương thức ứng viên và đối chiếu RVA

Các phương thức được kiểm tra gồm get_Coin, set_Coin, Awake, Start và StateCoin. Địa chỉ của get_Coin và set_Coin được tra trong thuộc tính Address và dump.cs. Với Awake, Start và StateCoin, địa chỉ được lấy từ Listing ở phần 6 và 7; RVA được tính bằng cách trừ Image Base 0x100000. Token metadata và RVA của mã IL trong DummyDll không phải địa chỉ mã native.

Phương thức	RVA	Địa chỉ Ghidra
get_Coin	0x22A0708	0x23A0708
set_Coin	0x22A0710	0x23A0710
Awake	0x22A15D0	0x23A15D0

Phương thức	RVA	Địa chỉ Ghidra
Start	0x22A15D4	0x23A15D4
StateCoin	0x22A16F8	0x23A16F8

get_Coin trả về int và không có tham số. set_Coin nhận int value, trả về void. Thân hàm rỗng hoặc return 0 trong DummyDll chỉ là phần được dựng để tra cứu; cần đọc mã native để biết hành vi thực tế.

```
// Methods

// RVA: 0x22A0708 Offset: 0x229C708 VA: 0x22A0708
public int get_Coin() { }

// RVA: 0x22A0710 Offset: 0x229C710 VA: 0x22A0710
public void set_Coin(int value) { }
```

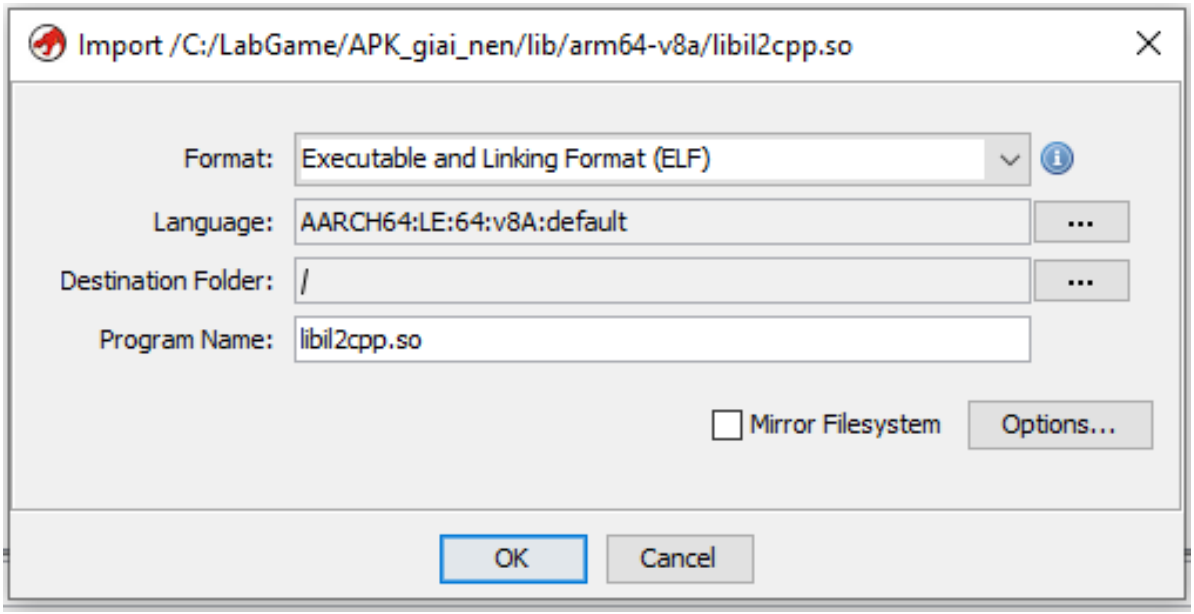
Hình 18. RVA và File Offset của get_Coin và set_Coin trong dump.cs

Hình 18 ghi File Offset của get_Coin là 0x229C708 và set_Coin là 0x229C710. Đây là vị trí trong file ELF. Với các phương thức còn lại, địa chỉ được đối chiếu trong Listing và RVA được tính bằng cách trừ Image Base 0x100000.

5. Import và gắn tên phương thức trong Ghidra

5.1. Import đúng thư viện ARM64

Project Non-Shared được tạo với tên dichnguoc và import file libil2cpp.so đã dùng với Il2CppDumper. Hộp import nhận dạng định dạng ELF và language AARCH64:LE:64:v8A:default. Sau khi mở CodeBrowser, bước phân tích tự động được chạy để nhận diện vùng mã, hàm và tham chiếu.



Hình 19. Import libil2cpp.so với định dạng ELF và ngôn ngữ AARCH64

5.2. Quy ước địa chỉ

Image Base trong lần import này là 0x100000. Ví dụ, get_Coin có RVA 0x22A0708 và địa chỉ Ghidra 0x23A0708. File Offset chỉ vị trí dữ liệu trong file ELF, còn địa chỉ Ghidra chỉ vị trí trong chương trình đã import.

Địa chỉ Ghidra = Image Base + RVA
0x100000 + 0x22A0708 = 0x23A0708

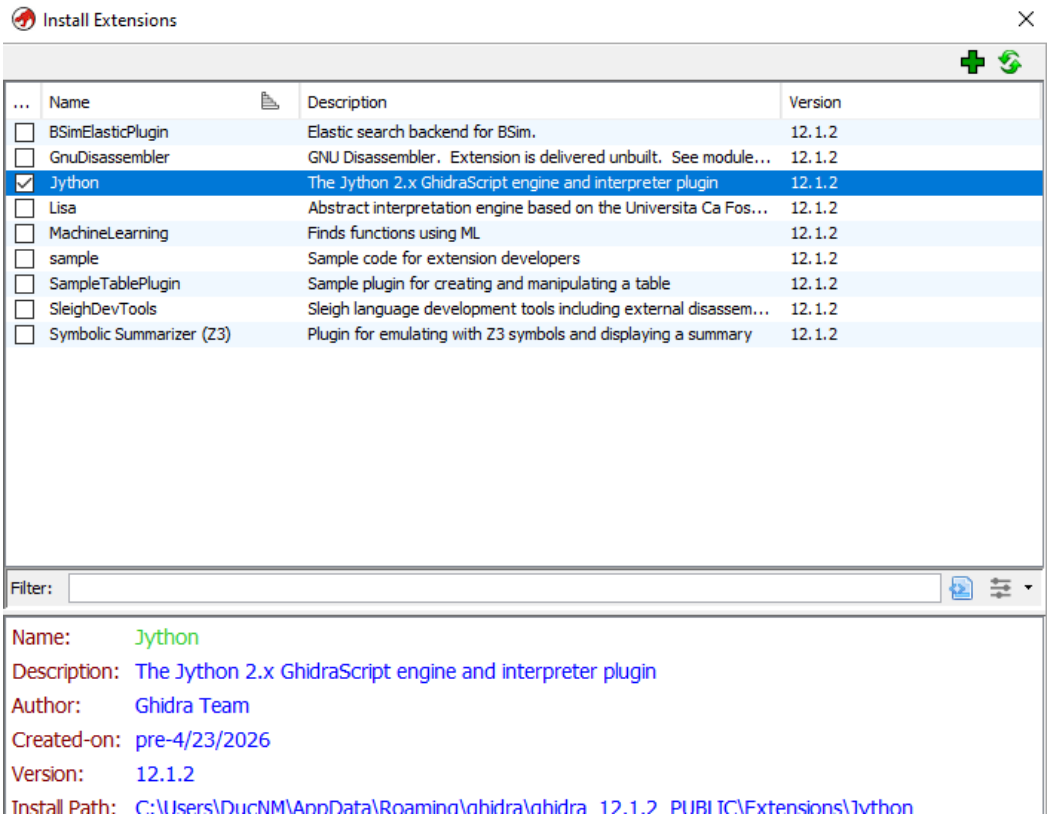
Listing hiển thị lệnh ARM64 và địa chỉ. Decompiler dựng mã giả để dễ đọc luồng xử lý, nhưng vẫn cần đối chiếu với khai báo dnSpy và các offset. Mã giả này không phải mã nguồn C++ hoặc C# gốc.

5.3 Khắc phục môi trường chạy script và hoàn tất gắn tên

Thư mục Il2CppDumper-win-v6.7.46 (1) được thêm vào Script Directories. Lần chạy đầu xuất hiện thông báo Ghidra was not started with PyGhidra. Python is not available, nên script chưa chạy được.

Để xử lý lỗi, Jython được bật trong File > Install Extensions, sau đó khởi động lại Ghidra. Script Il2CppDumper sử dụng Python 2/Jython, nên chỉ thị runtime được thêm vào đầu ghidra.py như sau:

```
# -*- coding: utf-8 -*-
# @runtime Jython
import json
```



Hình 20. Extension Jython phiên bản 12.1.2 được bật trong Ghidra

Sau khi chạy lại ghidra.py với script.json của cùng lần dump, Console hiển thị Script finished! và ghidra.py> Finished!. Tên các phương thức FoodSort xuất hiện trong Symbol Tree và project được lưu lại.

```
ghidra.py> Running...
Script finished!
ghidra.py> Finished!
```

Hình 21. Thông báo hoàn tất của lần chạy script thành công

6. Xác định nơi đọc và ghi field coin trong mã native

get_Coin tại 0x23A0708 chỉ gồm lệnh ldr w0, [x0, #0x20] rồi ret. Lệnh đọc giá trị 32 bit từ offset 0x20 của đối tượng và trả về; khớp private int _coin trong dnSpy. set_Coin tại 0x23A0710 thực hiện str w1, [x0, #0x20] rồi ret, ghi giá trị truyền vào cùng field.

```
                    ****************************************************************
                    *                          FUNCTION                            *
                    ****************************************************************
                      undefined FoodSort.PlayerResourceManager$$get_Coin()
     undefined        <UNASSIGNED>   <RETURN>
                    FoodSort.PlayerResourceManager$$get_Coin        XREF[2]:     01559c68, 016f4738(*)
023a0708 00 20 40 b9     ldr        w0,[x0, #0x20]
023a070c c0 03 5f d6     ret

                    ****************************************************************
                    *                          FUNCTION                            *
                    ****************************************************************
                      undefined FoodSort.PlayerResourceManager$$set_Coin()
     undefined        <UNASSIGNED>   <RETURN>
                    FoodSort.PlayerResourceManager$$set_Coin        XREF[2]:     01559c70, 016f4750(*)
023a0710 01 20 00 b9     str        w1,[x0, #0x20]
023a0714 c0 03 5f d6     ret
```

Hình 22. Listing của get_Coin và set_Coin xác nhận đọc và ghi offset 0x20

Hai lệnh xác nhận field _coin nằm ở offset 0x20 trong đối tượng PlayerResourceManager. 0x20 là độ lệch của field, không phải lượng coin hoặc địa chỉ cố định khi game chạy. Decompiler hiển thị undefined4 cho dữ liệu 4 byte, còn dnSpy xác định kiểu của field là int.

get_Coin và set_Coin chỉ đọc hoặc ghi field _coin. Trong đoạn mã đã xem, hai hàm không cộng coin và không gọi PlayerPrefs. Start được kiểm tra tiếp để tìm bước nạp số dư, còn StateCoin được kiểm tra để tìm bước cập nhật.

6.1. Loại trừ Awake khỏi luồng nạp coin

Awake tại 0x23A15D0 chỉ có lệnh ret, nên được loại khỏi ứng viên trực tiếp nạp hoặc khởi tạo coin trong bản thư viện này.

```
                     *************************************************************
                     *                          FUNCTION                         *
                     *************************************************************
                       undefined FoodSort.PlayerResourceManager$$Awake()
     undefined         <UNASSIGNED>   <RETURN>
                     FoodSort.PlayerResourceManager$$Awake          XREF[2]:     01559e10, 016f4db0(*)
023a15d0 c0 03 5f d6     ret
```

Hình 23. Awake của PlayerResourceManager chỉ trả về, không nạp coin

6.2. Logic coin khi vào game

Hàm xác định là FoodSort.PlayerResourceManager.Start, RVA 0x22A15D4, địa chỉ Ghidra 0x23A15D4. Ở dòng 18, hàm gọi UnityEngine.PlayerPrefs.GetInt với khóa _StringLiteral_3130 và giá trị mặc định 100. Dòng 19 ghi kết quả vào param_1 + 0x20, chính là field _coin đã đối chiếu.

```
                     *************************************************************
                       undefined FoodSort.PlayerResourceManager$$Start()
     undefined         <UNASSIGNED>   <RETURN>
                     FoodSort.PlayerResourceManager$$Start          XREF[2]:     015
023a15d4 fe 67 bc a9     stp        x30,x25,[sp, #-0x40]!
023a15d8 f8 5f 01 a9     stp        x24,x23,[sp, #0x10]
```

Decompile: FoodSort.PlayerResourceManager$$Start - (libil2cpp.so)

```
1
2  /* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
3
4  void FoodSort.PlayerResourceManager$$Start(long param_1)
5
```

```
18   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_3130,100,0);
19   *(undefined4 *)(param_1 + 0x20) = uVar1;
20   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_9554,3,0);
21   *(undefined4 *)(param_1 + 0x24) = uVar1;
22   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_4353,1,0);
23   *(undefined4 *)(param_1 + 0x28) = uVar1;
24   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_7786,3,0);
25   *(undefined4 *)(param_1 + 0x2c) = uVar1;
```

Hình 24. Start tại 0x23A15D4 đọc GetInt với mặc định 100 và ghi _coin ở +0x20

Start lấy số dư đã lưu bằng GetInt rồi ghi vào _coin. Khi khóa đã tồn tại, số coin được lấy từ lần lưu trước; khi chưa có khóa, giá trị mặc định là 100. Vì vậy, 100 không phải số coin được cộng lại mỗi lần vào game. Các lời gọi GetInt tại offset 0x24, 0x28 và 0x2C thuộc các field booster.

6.3. Đối chiếu khóa lưu và kết luận luồng khởi tạo

Khi mở _StringLiteral_3130 trong Listing, nhãn có chú thích Coin_SandSort. Start dùng khóa này để đọc bằng GetInt, còn StateCoin dùng cùng khóa để lưu bằng SetInt.

```
                    StringLiteral_3130                              XREF[1]:     04e85108(*)
04fbe488 75             ??          75h    u                                    Coin_SandSort
04fbe489 18             ??          18h
04fbe48a 00             ??          00h
04fbe48b a0             ??          A0h
04fbe48c 00             ??          00h
04fbe48d 00             ??          00h
04fbe48e 00             ??          00h
04fbe48f 00             ??          00h
```

Hình 25. Nhãn StringLiteral_3130 mang nội dung Coin_SandSort

Phần nạp coin có thể diễn giải bằng đoạn mã dưới đây. Đây là cách viết lại hành vi đã quan sát, không phải mã nguồn gốc:

_coin = PlayerPrefs.GetInt("Coin_SandSort", 100);

7. Logic cập nhật coin sau khi nhận thưởng

StateCoin thực hiện việc cộng hoặc trừ số dư theo tham số truyền vào. Nhánh nhận thưởng thông thường trên UIWinGame gọi hàm này với state = true và coin = 100. Phần phân tích dưới đây đối chiếu cả thao tác nhận thưởng và luồng kiểm tra thắng dẫn đến việc mở giao diện này.

7.1. Hàm StateCoin cộng trừ và lưu số dư

FoodSort.PlayerResourceManager.StateCoin(bool state, int coin, string placement = "unknown", string groupPlacement = "unknown") có RVA 0x22A16F8, địa chỉ Ghidra 0x23A16F8. param_1 là đối tượng; param_2 là bool state; param_3 là int coin. Hai tham số được lưu vào đối tượng phụ trước khi sử dụng; số dư vẫn nằm ở param_1 +0x20.

Hình 26 cho thấy phép tính cộng trừ và bước lưu số dư trong StateCoin.

Dòng 39 khởi tạo số thay đổi bằng -coin. Dòng 40 - 41 chuyển thành +coin nếu state khác 0. Dòng 43 cộng số thay đổi vào coin hiện tại; dòng 44 ghi lại _coin. Dòng 45 gọi PlayerPrefs.SetInt với khóa _StringLiteral_3130; dòng 46 gọi PlayerPrefs.Save.

Decompile: FoodSort.PlayerResourceManager$$StateCoin - (libil2cpp.so)

```
25    FUN_021f8840(&Method$FoodSort.PlayerResourceManager.<>c__DisplayClass100_0.<StateCoin>b__0());
26    FUN_021f8840(&FoodSort.PlayerResourceManager.<>c__DisplayClass100_0_TypeInfo);
27    FUN_021f8840(&StringLiteral_3130);
28    FUN_021f8840(&StringLiteral_6024);
29    bRam00000000051c39a8 = 1;
30  }
31  lVar2 = thunk_FUN_02203abc(_FoodSort.PlayerResourceManager.<>c__DisplayClass100_0_TypeInfo);
32  System.Object$$.ctor(lVar2,0);
33  if (lVar2 != 0) {
34    *(undefined8 *)(lVar2 + 0x18) = param_4;
35    *(byte *)(lVar2 + 0x10) = param_2 & 1;
36    *(undefined4 *)(lVar2 + 0x14) = param_3;
37    thunk_FUN_02255370((undefined8 *)(lVar2 + 0x18),param_4);
38    if (param_1 != 0) {
39      iVar1 = -*(int *)(lVar2 + 0x14);
40      if (*(char *)(lVar2 + 0x10) != '\0') {
41        iVar1 = *(int *)(lVar2 + 0x14);
42      }
43      iVar1 = iVar1 + *(int *)(param_1 + 0x20);
44      *(int *)(param_1 + 0x20) = iVar1;
45      UnityEngine.PlayerPrefs$$SetInt(_StringLiteral_3130,iVar1,0);
46      UnityEngine.PlayerPrefs$$Save(0);
```

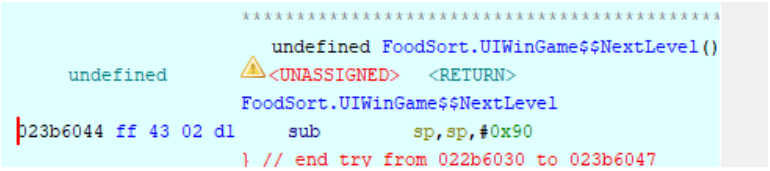
Hình 26. Nhánh cộng trừ, cập nhật field và lưu coin trong StateCoin

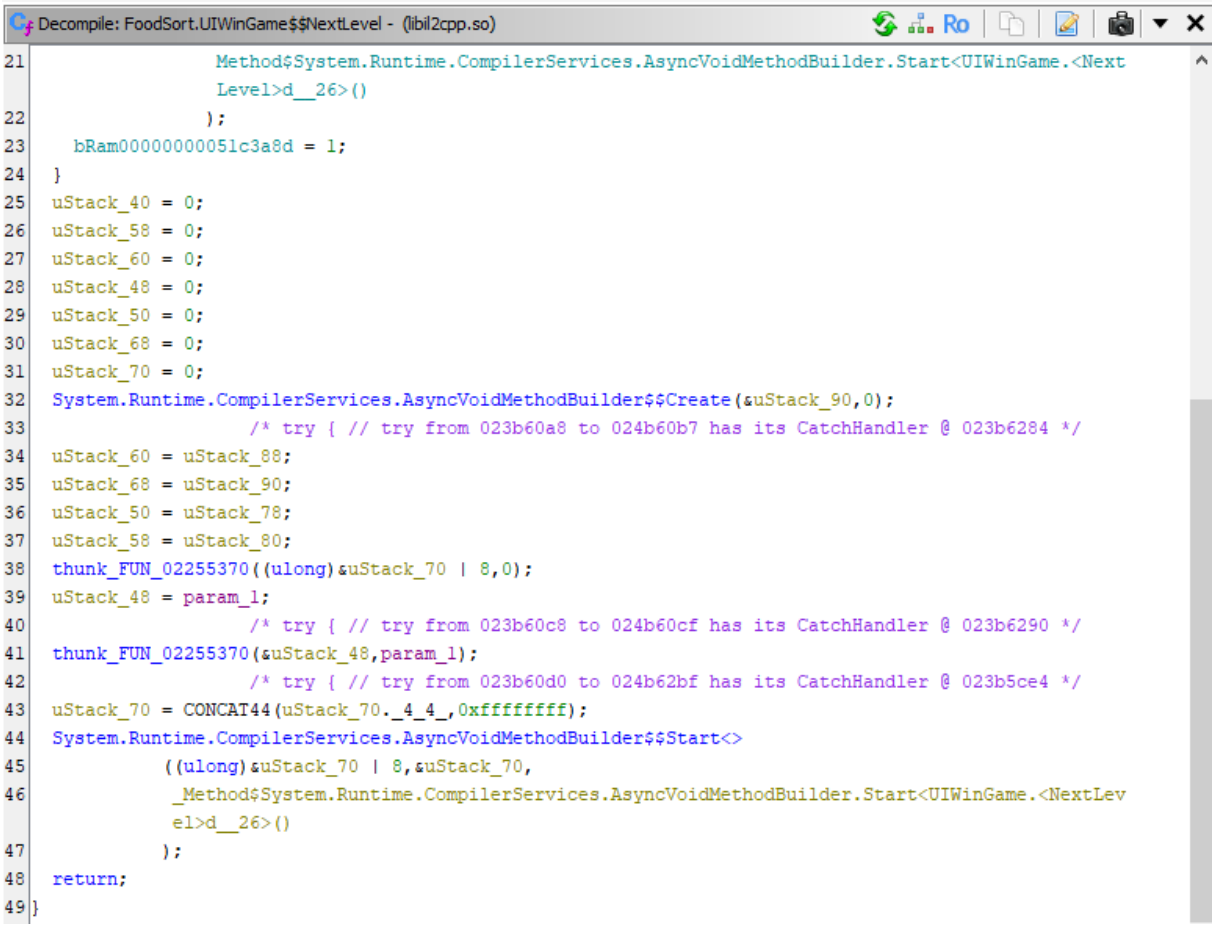
```
int change = state ? coin : -coin;
_coin = _coin + change;
PlayerPrefs.SetInt("Coin_SandSort", _coin);
PlayerPrefs.Save();
```

Với coin là số dương, state = true làm tăng số dư, còn state = false làm giảm số dư. Phía sau có các lời gọi RecordLevelCoinChange và TrackWhenMetadataReady. Phần xử lý này chưa được phân tích đầy đủ nên chưa xác định có bước xác thực số dư trên máy chủ hay không.

7.2. Nhánh thưởng 100 coin trong luồng NextLevel

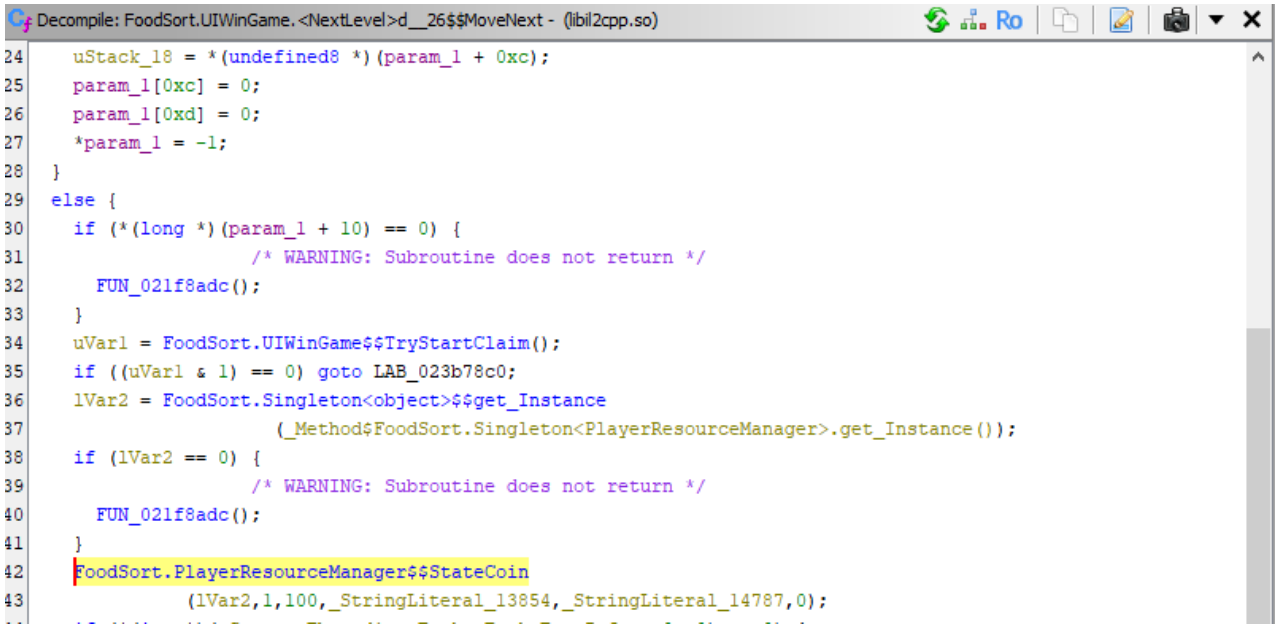
UIWinGame.NextLevel tại 0x23B6044, RVA 0x22B6044, chuẩn bị state machine <NextLevel>d__26 và gọi AsyncVoidMethodBuilder.Start. Phần xử lý nhận thưởng nằm trong <NextLevel>d__26.MoveNext. Trong nhánh này, TryStartClaim phải trả về true trước khi lấy PlayerResourceManager và gọi StateCoin.





Hình 27. NextLevel tại 0x23B6044 khởi chạy state machine NextLevel d__26

Dòng 42–43 trong Hình 28 truyền state = 1 và coin = 100 vào StateCoin. Giá trị này được đối chiếu trong Listing bằng các lệnh mov w1, #0x1 và mov w2, #0x64; 0x64 tương đương 100. Lệnh gọi StateCoin nằm tại 0x23B7804, không phải địa chỉ bắt đầu NextLevel hoặc MoveNext.



Hình 28. MoveNext kiểm tra TryStartClaim rồi gọi StateCoin với lượng thưởng 100

Hai tham số chuỗi tại lời gọi có nhãn _StringLiteral_13854 và _StringLiteral_14787. Nội dung chuỗi chưa được xác định nên chưa biết tên nguồn thưởng. Giá trị 100 ở lời gọi này là lượng coin được cộng, còn 100 trong Start là giá trị mặc định khi chưa có dữ liệu lưu.

7.3. Điều kiện bắt đầu nhận thưởng

FoodSort.UIWinGame.TryStartClaim có địa chỉ Ghidra 0x23B632C, RVA 0x22B632C. Hàm đọc byte ở offset 0xA8 vào cVar1. Nếu giá trị ban đầu bằng 0, phép ghi undefined2 tại +0xA8 bằng 1 đặt _isClaimingReward ở +0xA8 thành true và _isRewardAdCallbackHandled ở +0xA9 thành false. Hai tên field được đối chiếu bằng dnSpy ở mục 7.4. Cuối hàm trả về cVar1 == 0 dựa trên giá trị cờ ban đầu.

```
                    undefined FoodSort.UIWinGame$$TryStartClaim()
     undefined      <UNASSIGNED>   <RETURN>
                    FoodSort.UIWinGame$$TryStartClaim              XREF[4]:     015
                                                                                Foc
                                                                                Foc
023b632c fe 57 be a9     stp         x30,x21,[sp, #-0x20]!
023b6330 f4 4f 01 a9     stp         x20,x19,[sp, #0x10]
023b6334 74 70 01 b0     adrp        x20,0x51c3000
```

Decompile: FoodSort.UIWinGame$$TryStartClaim - (libil2cpp.so)

```
11
12  if ((bRam00000000051c3a90 & 1) == 0) {
13    FUN_021f8840(&UnityEngine.Object_TypeInfo);
14    bRam00000000051c3a90 = 1;
15  }
16  cVar1 = *(char *)(param_1 + 0xa8);
17  if (cVar1 == '\0') {
18    *(undefined2 *)(param_1 + 0xa8) = 1;
19    if (*(long *)(param_1 + 0x50) == 0) {
20LAB_023b63d4:
21                    /* WARNING: Subroutine does not return */
22      FUN_021f8adc();
23    }
24    puVar2 = (undefined8 *)(*(long *)(param_1 + 0x50) + 0x20);
25    *puVar2 = 0;
26    thunk_FUN_02255370(puVar2,0);
27    uVar4 = *(undefined8 *)(param_1 + 0x58);
28    if (*(int *)(_UnityEngine.Object_TypeInfo + 0xe0) == 0) {
29      thunk_FUN_021e5a80();
30    }
31    uVar3 = UnityEngine.Object$$op_Inequality(uVar4,0,0);
32    if ((uVar3 & 1) != 0) {
33      if (*(long *)(param_1 + 0x58) == 0) goto LAB_023b63d4;
34      puVar2 = (undefined8 *)(*(long *)(param_1 + 0x58) + 0x20);
35      *puVar2 = 0;
36      thunk_FUN_02255370(puVar2,0);
37    }
38  }
39  return cVar1 == '\0';
40}
41
```

Hình 29. TryStartClaim tại 0x23B632C kiểm tra cờ và xóa callback của các nút

Sau khi bắt đầu nhận thưởng, dòng 24–26 ghi 0 vào field +0x20 của UIClick tại this +0x50. Dòng 33–36 làm tương tự với UIClick tại this +0x58 nếu đối tượng tồn tại. Đối chiếu mục 7.5 cho thấy đây là thao tác xóa _action của nút tiếp tục và nút X2. Việc xóa _action khiến các nút không còn callback để tiếp tục kích hoạt trong lúc nhận thưởng.

Nếu _isClaimingReward ban đầu là false, TryStartClaim cho phép luồng nhận thưởng tiếp tục. Khi cờ đã là true, hàm trả về false và nhánh cộng coin bị bỏ qua. Vì vậy, hai lần gọi liên tiếp khi cờ chưa được đặt lại không đều nhận thưởng. Kết quả này áp dụng khi hàm hoàn tất bình thường; trường hợp gặp đối tượng null có thể đi vào nhánh xử lý lỗi.

7.4. Đối chiếu tên cờ và nơi đặt lại

Trong dnSpy, _isClaimingReward có kiểu bool và FieldOffset 0xA8, khớp vị trí được đọc trong TryStartClaim. Field _isRewardAdCallbackHandled nằm ở offset 0xA9 và là một cờ riêng.

```
215         // Token: 0x04000BDD RID: 3037
216         [Token(Token = "0x4000BDD")]
217         [FieldOffset(Offset = "0xA8")]
218         private bool _isClaimingReward;
219
220         // Token: 0x04000BDE RID: 3038
221         [Token(Token = "0x4000BDE")]
222         [FieldOffset(Offset = "0xA9")]
223         private bool _isRewardAdCallbackHandled;
```

Hình 30. Field _isClaimingReward ở offset 0xA8 trong UIWinGame

UIWinGame.OnEnable có RVA 0x22B5B40, địa chỉ Ghidra 0x23B5B40. Khi UIManager tồn tại, dòng 71 ghi 0 bằng kiểu undefined2 tại +0xA8, đặt lại cả _isClaimingReward (+0xA8) và _isRewardAdCallbackHandled (+0xA9). Dòng 72 gọi BindClaimActions để gắn lại callback. Phần đã kiểm tra chỉ cho thấy cách đặt lại cờ trong OnEnable, chưa xác định số lần giao diện này được mở trong một level.

```
                    undefined FoodSort.UIWinGame$$OnEnable()
     undefined      <UNASSIGNED>   <RETURN>
                    FoodSort.UIWinGame$$OnEnable
023b5b40 e8 0f 1b fc     str        d8,[sp, #-0x50]!
023b5b44 fe 07 00 f9     str        x30,[sp, #0x8]
```

Decompile: FoodSort.UIWinGame$$OnEnable - (libil2cpp.so)

```
54                          (*(undefined4 *)(lVar1 + 0xc),*(undefined4 *)(lVar1 + 0x10),
55                           *(undefined4 *)(lVar1 + 0x14),0x3f19999a,uVar5,0);
56          uVar5 = DG.Tweening.TweenSettingsExtensions$$SetEase<object>
57                          (uVar5,0x1b,
58                           _Method$DG.Tweening.TweenSettingsExtensions.SetEase<TweenerCore<Vector3,-
                             Vector3,-VectorOptions>>()
59                          );
60          *(undefined8 *)(param_1 + 0x98) = uVar5;
61          thunk_FUN_02255370((long *)(param_1 + 0x98),uVar5);
62          uVar5 = DG.Tweening.DOTweenModuleUI$$DOFade
63                          (0x3f4ccccd,0x3f19999a,*(undefined8 *)(param_1 + 0x40),0);
64          *(undefined8 *)(param_1 + 0xa0) = uVar5;
65                    /* catch() { ... } // from try @ 023b60d0 with catch @ 023b5ce4 */
66          thunk_FUN_02255370((long *)(param_1 + 0xa0),uVar5);
67          lVar1 = FoodSort.Singleton<object>$$get_Instance
68                          (_Method$FoodSort.Singleton<UIManager>.get_Instance());
69          if (lVar1 != 0) {
70            FoodSort.UIManager$$ShowCoin(lVar1,0);
71            *(undefined1 *)(param_1 + 0xa8) = 0;
72            FoodSort.UIWinGame$$BindClaimActions(param_1);
73            if ((bRam0000000051c3a8c & 1) == 0) {
74              FUN_021f8840(&
75                           Method$System.Runtime.CompilerServices.AsyncTaskMethodBuilder.Start<UIWinGa
```

Hình 31. OnEnable tại 0x23B5B40 đặt lại hai cờ và gọi BindClaimActions

7.5. Gắn callback cho các nút nhận thưởng

Trong FoodSort.UIWinGame, dnSpy xác nhận _uIClickNextLevel có FieldOffset 0x50 và _uIClickRewardX2 có FieldOffset 0x58, đều thuộc kiểu UIClick. Đây là các đối tượng được truy cập trong BindClaimActions và TryStartClaim.

```
// Token: 0x04000BD2 RID: 3026
[Token(Token = "0x4000BD2")]
[FieldOffset(Offset = "0x50")]
[SerializeField]
private UIClick _uIClickNextLevel;

// Token: 0x04000BD3 RID: 3027
[Token(Token = "0x4000BD3")]
[FieldOffset(Offset = "0x58")]
[SerializeField]
private UIClick _uIClickRewardX2;
```

Hình 32. Các field UIClick của UIWinGame ở offset 0x50 và 0x58

Trong FoodSort.UIClick, field private Action _action có FieldOffset 0x20. Phép ghi tại UIClick +0x20 vì vậy là cập nhật callback của nút.

```
// Token: 0x04000B43 RID: 2883
[Token(Token = "0x4000B43")]
[FieldOffset(Offset = "0x20")]
private Action _action;
```

Hình 33. Field UIClick._action kiểu Action tại offset 0x20

BindClaimActions đăng ký NextLevel và ClaimX2Reward

FoodSort.UIWinGame.BindClaimActions bắt đầu tại 0x23B5D30, tương ứng RVA 0x22B5D30 với Image Base 0x100000. Dòng 22 lấy UIClick tại this +0x50. Dòng 24 tạo System.Action gắn đối tượng UIWinGame hiện tại với NextLevel, rồi dòng 27 ghi Action vào UIClick +0x20. Ở bước này, NextLevel được gắn làm callback của nút và sẽ chạy khi callback được kích hoạt.

```
                    undefined FoodSort.UIWinGame$$BindClaimActions()
     undefined      <UNASSIGNED>   <RETURN>
                    FoodSort.UIWinGame$$BindClaimActions            XREF

023b5d30 fe 5f bd a9     stp        x30,x23,[sp, #-0x30]!
```

Decompile: FoodSort.UIWinGame$$BindClaimActions - (libil2cpp.so)

```
1
2  /* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
3
4  void FoodSort.UIWinGame$$BindClaimActions(long param_1)
5
6  {
7    ulong *puVar1;
8    char cVar2;
9    bool bVar3;
10   undefined8 uVar4;
11   ulong uVar5;
12   long lVar6;
13   undefined8 *puVar7;
14
15   if ((bRam000000000051c3a91 & 1) == 0) {
16     FUN_021f8840(&System.Action_TypeInfo);
17     FUN_021f8840(&UnityEngine.Object_TypeInfo);
18     FUN_021f8840(&Method$FoodSort.UIWinGame.ClaimX2Reward());
19     FUN_021f8840(&Method$FoodSort.UIWinGame.NextLevel());
20     bRam000000000051c3a91 = 1;
21   }
22   lVar6 = *(long *)(param_1 + 0x50);
23   uVar4 = thunk_FUN_02203abc(_System.Action_TypeInfo);
24   System.Action$$.ctor(uVar4,param_1,_Method$FoodSort.UIWinGame.NextLevel(),0);
25   if (lVar6 != 0) {
26     puVar7 = (undefined8 *)(lVar6 + 0x20);
27     *puVar7 = uVar4;
28     thunk_FUN_02255370(puVar7,uVar4);
29     uVar4 = *(undefined8 *)(param_1 + 0x58);
30     if (*(int *)(_UnityEngine.Object_TypeInfo + 0xe0) == 0) {
31       thunk_FUN_021e5a80();
```

Hình 34. BindClaimActions tại 0x23B5D30 gắn NextLevel vào nút _uIClickNextLevel

Với UIClick tại this +0x58, hàm kiểm tra đối tượng UnityEngine.Object khác null rồi tạo System.Action gắn với ClaimX2Reward ở dòng 39. Action được ghi vào field _action tại +0x20. ClaimX2Reward được xác định là callback của nút X2, nhưng phần xử lý quảng cáo và lượng coin được cộng chưa được phân tích.



Hình 35. BindClaimActions đăng ký ClaimX2Reward cho _uIClickRewardX2

7.6. Sự kiện nhả chuột thực thi callback của UIClick

FoodSort.UIClick.OnPointerUp(PointerEventData) được khai báo trong dnSpy với thuộc tính Address: RVA 0x22B11BC và File Offset 0x22AD1BC. Trong Ghidra hiện tại, địa chỉ bắt đầu tương ứng là 0x23B11BC. RVA nhỏ ở chú thích mã IL của DummyDll không phải địa chỉ native này.

Ở dòng 48, OnPointerUp gọi RectangleContainsScreenPoint để kiểm tra điểm nhả con trỏ nằm trong vùng hình chữ nhật của nút. Dòng 50 yêu cầu kết quả đúng và _action đọc từ this +0x20 khác null. Dòng 53 thực hiện lời gọi gián tiếp thông qua delegate đó. Đối chiếu với BindClaimActions xác nhận: với đối tượng _uIClickNextLevel, _action đã được gắn với UIWinGame.NextLevel, nên nhả con trỏ hợp lệ sẽ đi vào NextLevel.

```
                    undefined FoodSort.UIClick$$OnPointerUp()
     undefined      <UNASSIGNED>   <RETURN>
                    FoodSort.UIClick$$OnPointerUp                XREF[2
023b11bc e9 23 bc 6d     stp        d9,d8,[sp, #-0x40]!
023b11c0 fe 0b 00 f9     str        x30,[sp, #0x10]
023b11c4 f6 57 02 a9     stp        x22,x21,[sp, #0x20]
```

Decompile: FoodSort.UIClick$$OnPointerUp - (libil2cpp.so)

```
44  }
45  else if (*plVar3 != _UnityEngine.RectTransform_TypeInfo) {
46    plVar3 = (long *)0x0;
47  }
48  uVar4 = UnityEngine.RectTransformUtility$$RectangleContainsScreenPoint(uVar5,uVar6,plVar3,uVa
49  ;
50  if (((uVar4 & 1) != 0) && (lVar1 = *(long *)(param_1 + 0x20), lVar1 != 0)) {
51                    /* WARNING: Could not recover jumptable at 0x023b1314. Too many branches */
52                    /* WARNING: Treating indirect jump as call */
53    (**(code **)(lVar1 + 0x18))(*(undefined8 *)(lVar1 + 0x40),*(undefined8 *)(lVar1 + 0x28));
54    return;
55  }
56  return;
57}
```

Hình 36. OnPointerUp tại 0x23B11BC kiểm tra vị trí con trỏ và gọi _action

OnPointerUp thực thi callback qua delegate nên tại dòng gọi không xuất hiện trực tiếp tên NextLevel.

7.7. Điều kiện thắng và mở giao diện nhận thưởng

FoodSort.LevelManager.TryWinLevel bắt đầu tại địa chỉ Ghidra 0x239B76C. Hàm kiểm tra byte tại this +0xF2 đang bằng 0 rồi gọi CheckWin. Khi CheckWin trả về true, lệnh b tại 0x239B78C chuyển đến CompleteLevelAsWin ở 0x239B2CC. Đây là tham chiếu mã được tìm từ danh sách References của CompleteLevelAsWin; các dòng tham chiếu dữ liệu trong danh sách không phải lời gọi thực thi.

Decompile: FoodSort.LevelManager$$TryWinLevel - (libil2cpp.so)

```
1
2 /* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
3
4 void FoodSort.LevelManager$$TryWinLevel(long param_1)
5
6 {
7   ulong uVar1;
8   undefined8 uStack_90;
9   undefined8 uStack_88;
10  undefined8 uStack_80;
11  undefined8 uStack_78;
12  undefined8 uStack_70;
13  undefined8 uStack_68;
14  undefined8 uStack_60;
15  undefined8 uStack_58;
16  undefined8 uStack_50;
17  long alStack_48 [5];
18
19  if ((*(char *)(param_1 + 0xf2) == '\0') &&
20     (uVar1 = FoodSort.LevelManager$$CheckWin(), (uVar1 & 1) != 0)) {
21    if ((bRam0000000051c3937 & 1) == 0) {
22      FUN_021f8840(&
23                    Method$System.Runtime.CompilerServices.AsyncVoidMethodBuilder.Start<LevelManager.
                      <CompleteLevelAsWin>d__205>()
```

Hình 37. TryWinLevel kiểm tra cờ thắng và kết quả CheckWin trước khi xử lý thắng

CheckWin bắt đầu tại 0x239BDC4 và chỉ xét tiếp khi byte tại this +0xF3 bằng 0. Hàm có nhánh theo IsNoHolesNormalLevel, duyệt danh sách BottleController và gọi IsEmpty. Những đoạn khác kiểm tra Header.CheckAnyBottle, IsAnyBottleFlying và CheckCompleteAllBottles. Đoạn cuối còn so sánh giá trị tại this +0x104 với số phần tử của danh sách tại this +0x148. Vì các trường này và thân các hàm con chưa được đối chiếu hết, điều kiện thắng chưa được rút gọn thành một quy tắc duy nhất. Riêng dòng 119 trong Hình 38 kiểm tra kết quả CheckCompleteAllBottles bằng 0, nên không thể chỉ dựa vào tên hàm để kết luận tất cả chai hoàn thành thì thắng.

```
114                     /* try { // try from 0239bf20 to 0249bf23 has its CatchHandler @ 0239c6f8 */
115     uVar1 = FoodSort.LevelManager$$IsAnyBottleFlying(param_1);
116                     /* try { // try from 0239bf28 to 0249bf2b has its CatchHandler @ 0239c6c0 */
117                     /* try { // try from 0239bf34 to 0249bf3b has its CatchHandler @ 0239c698 */
118     if (((uVar1 & 1) == 0) &&
119        (uVar1 = FoodSort.LevelManager$$CheckCompleteAllBottles(param_1), (uVar1 & 1) != 0)) {
120       iVar2 = 0;
121       if (*(long *)(param_1 + 0x148) != 0) {
122         iVar2 = *(int *)(*(long *)(param_1 + 0x148) + 0x18);
123       }
124                     /* try { // try from 0239bf4c to 0249bf63 has its CatchHandler @ 0239c6f8 */
125       return *(int *)(param_1 + 0x104) <= iVar2;
126     }
127   }
128                     /* catch() { ... } // from try @ 0239bcdc with catch @ 0239be18 */
129   return false;
130 }
```

Hình 38. Đoạn cuối CheckWin kiểm tra các hàm con và phép so sánh trước khi trả kết quả

Trong dnSpy, LevelManager có field bool _isWin ở offset 0xF2 và bool _isLose ở offset 0xF3. Hai khai báo này xác nhận tên của các cờ được đọc trong TryWinLevel, CheckWin và CompleteLevelAsWin.MoveNext. Field int _levelDisplay ở offset 0xF4 cũng xuất hiện trong cùng ảnh; phép tăng giá trị tại offset này trong luồng thắng là cập nhật giá trị màn chơi, không phải tăng coin.

LevelManager

```
1327        private GameManager _gameManager;
1328
1329        // Token: 0x040009CB RID: 2507
1330        [Token(Token = "0x40009CB")]
1331        [FieldOffset(Offset = "0xE8")]
1332        private TutorialManager _tutorialManager;
1333
1334        // Token: 0x040009CC RID: 2508
1335        [Token(Token = "0x40009CC")]
1336        [FieldOffset(Offset = "0xF0")]
1337        private bool _isStartGame;
1338
1339        // Token: 0x040009CD RID: 2509
1340        [Token(Token = "0x40009CD")]
1341        [FieldOffset(Offset = "0xF1")]
1342        private bool _isBoardReady;
1343
1344        // Token: 0x040009CE RID: 2510
1345        [Token(Token = "0x40009CE")]
1346        [FieldOffset(Offset = "0xF2")]
1347        [SerializeField]
1348        private bool _isWin;
1349
1350        // Token: 0x040009CF RID: 2511
1351        [Token(Token = "0x40009CF")]
1352        [FieldOffset(Offset = "0xF3")]
1353        [SerializeField]
1354        private bool _isLose;
1355
1356        // Token: 0x040009D0 RID: 2512
1357        [Token(Token = "0x40009D0")]
1358        [FieldOffset(Offset = "0xF4")]
1359        private int _levelDisplay;
1360
```

Hình 39. Các field _isWin, _isLose và _levelDisplay của LevelManager trong dnSpy

CompleteLevelAsWin ở 0x239B2CC khởi chạy state machine <CompleteLevelAsWin>d__205. Phần xử lý nằm trong MoveNext tại 0x239E110. Để xác định đối tượng được bật trong hàm này, field tại LevelManager +0x68 được đối chiếu trong dnSpy: đó là private UIWinGame _uIWinGame, có thuộc tính SerializeField.

```
LevelManager
1240        private Transform _holesParent;
1241
1242        // Token: 0x040009B9 RID: 2489
1243        [Token(Token = "0x40009B9")]
1244        [FieldOffset(Offset = "0x58")]
1245        [SerializeField]
1246        private Hole _holePrefab;
1247
1248        // Token: 0x040009BA RID: 2490
1249        [Token(Token = "0x40009BA")]
1250        [FieldOffset(Offset = "0x60")]
1251        [SerializeField]
1252        private Hole[] _holes;
1253
1254        // Token: 0x040009BB RID: 2491
1255        [Token(Token = "0x40009BB")]
1256        [FieldOffset(Offset = "0x68")]
1257        [SerializeField]
1258        private UIWinGame _uIWinGame;
1259
1260        // Token: 0x040009BC RID: 2492
1261        [Token(Token = "0x40009BC")]
1262        [FieldOffset(Offset = "0x70")]
1263        [SerializeField]
1264        private UILoseGame _uILoseGame;
1265
1266        // Token: 0x040009BD RID: 2493
1267        [Token(Token = "0x40009BD")]
1268        [FieldOffset(Offset = "0x78")]
1269        [SerializeField]
1270        private UIRateUs _uIRateUs;
```

Hình 40. Field _uIWinGame thuộc kiểu UIWinGame tại offset 0x68 của LevelManager

Ở nhánh bắt đầu của MoveNext, nếu _isWin hoặc _isLose đã là true thì hàm chuyển đến phần kết thúc. Nếu cả hai cờ đều false, hàm đặt _isWin thành true và gọi Task.Delay với giá trị 500 ms. Điều kiện này giúp tránh bắt đầu lại phần xử lý thắng khi màn chơi đã có trạng thái kết thúc.

```
48  else {
49    if (lVar9 == 0) {
50                    /* WARNING: Subroutine does not return */
51      FUN_021f8adc();
52    }
53    if ((*(char *)(lVar9 + 0xf2) != '\0') || (*(char *)(lVar9 + 0xf3) != '\0')) goto LAB_0239e578;
54    *(undefined1 *)(lVar9 + 0xf2) = 1;
55    if (*(int *)(_System.Threading.Tasks.Task_TypeInfo + 0xe0) == 0) {
56      thunk_FUN_021e5a80();
57    }
58    lVar3 = System.Threading.Tasks.Task$$Delay(500,0);
59    if (lVar3 == 0) {
60                    /* WARNING: Subroutine does not return */
61      FUN_021f8adc();
62    }
```

Hình 41. MoveNext kiểm tra hai cờ, đánh dấu thắng và chờ 500 ms

Sau khi chờ, dòng 81–87 lấy đối tượng tại LevelManager +0x68, gọi Component.get_gameObject rồi GameObject.SetActive với giá trị true nếu đối tượng tồn tại. Kết hợp với Hình 40, đoạn này xác nhận CompleteLevelAsWin bật GameObject của UIWinGame. Đoạn mã mở giao diện chưa trực tiếp cộng coin; thao tác cộng 100 nằm trong nhánh NextLevel.MoveNext đã phân tích ở mục 7.2.

```
}
System.Runtime.CompilerServices.TaskAwaiter$$GetResult(&uStack_38,0);
if (lVar9 == 0) {
                    /* WARNING: Subroutine does not return */
  FUN_021f8adc();
}
if (*(long *)(lVar9 + 0x68) != 0) {
  lVar3 = UnityEngine.Component$$get_gameObject(*(long *)(lVar9 + 0x68),0);
  if (lVar3 == 0) {
                    /* WARNING: Subroutine does not return */
    FUN_021f8adc();
  }
  UnityEngine.GameObject$$SetActive(lVar3,1,0);
}
lVar3 = *(long *)(lVar9 + 0x98);
if (lVar3 != 0) {
  (**(code **)(lVar3 + 0x18))
```

Hình 42. MoveNext lấy GameObject của _uiWinGame tại offset 0x68 và gọi SetActive true

7.8. Tổng hợp luồng và giới hạn chứng minh

Khi _isWin chưa được đặt và CheckWin trả về true, TryWinLevel chuyển sang CompleteLevelAsWin. Phần MoveNext của luồng này kiểm tra _isWin và _isLose, đánh dấu thắng, chờ 500 ms rồi bật GameObject của _uiWinGame nếu đối tượng tồn tại. Khi giao diện được kích hoạt, OnEnable có nhánh đặt lại hai cờ nhận thưởng và gọi BindClaimActions nếu UIManager tồn tại.

Sau khi callback đã được gắn, thao tác nhả con trỏ hợp lệ trên nút tiếp tục làm OnPointerUp gọi NextLevel. NextLevel khởi chạy state machine; phần MoveNext kiểm tra TryStartClaim và gọi StateCoin với state = true, coin = 100 khi được phép nhận thưởng. StateCoin cộng 100 vào _coin, lưu số dư mới bằng PlayerPrefs.SetInt với khóa Coin_SandSort rồi gọi PlayerPrefs.Save.

Vị trí	**Địa chỉ Ghidra**	**Ý nghĩa**
TryWinLevel	0x239B76C	Kiểm tra chưa thắng và gọi CheckWin; chuyển đến CompleteLevelAsWin khi điều kiện đúng
CheckWin	0x239BDC4	Trả kết quả kiểm tra thắng theo các nhánh đã khảo sát
CompleteLevelAsWin	0x239B2CC	Khởi chạy state machine xử lý thắng
MoveNext của CompleteLevelAsWin	0x239E110	Đánh dấu thắng, chờ 500 ms và bật GameObject của UIWinGame
NextLevel	0x23B6044	Khởi chạy luồng bất đồng bộ
TryStartClaim	0x23B632C	Kiểm tra cờ nhận thưởng và xóa callback trong nhánh bắt đầu
Lời gọi StateCoin	0x23B7804	Truyền state = true và coin = 100

Vị trí	Địa chỉ Ghidra	Ý nghĩa
StateCoin	0x23A16F8	Cộng/trừ và lưu số dư
OnEnable	0x23B5B40	Đặt lại hai cờ và gắn lại callback trong nhánh đã xem
BindClaimActions	0x23B5D30	Gắn NextLevel và ClaimX2Reward vào _action của các nút
UIClick.OnPointerUp	0x23B11BC	Kiểm tra điểm nhả và gọi delegate _action

Lượng thưởng 100 thuộc nhánh nhận thưởng qua nút tiếp tục, không được cộng tự động ngay khi CheckWin trả về true.

8. Nhận xét an toàn và giới hạn phân tích tĩnh

8.1. Kịch bản can thiệp tài nguyên phía client

Một kịch bản có thể xảy ra là thay đổi dữ liệu lưu cục bộ của khóa Coin_SandSort trước khi game nạp lại. Nếu thay đổi có hiệu lực và không bị cơ chế khác kiểm tra hoặc ghi đè, Start sẽ đọc số đã thay đổi rồi đưa vào _coin. Kịch bản thứ hai là can thiệp lượng thưởng truyền vào StateCoin hoặc thao tác cập nhật số dư, từ đó làm tăng tài nguyên ngoài mức mà nhánh thưởng thông thường quyết định.

Chưa có thử nghiệm can thiệp để kiểm tra tác động thực tế của hai kịch bản này.

8.2 Tác động và giới hạn của kết luận

Nếu số dư phía client bị thay đổi, phần hiển thị và các thao tác tiêu coin có thể bị ảnh hưởng khi chúng sử dụng số dư này. Các ảnh hiện có chưa cho thấy máy chủ có chấp nhận số dư đã sửa hay số coin đó có dùng được cho mọi chức năng hay không.

Dữ liệu PlayerPrefs khi game chạy và số coin hiển thị sau khi nhận thưởng chưa được kiểm tra. Các nhánh của CheckWin và một số field chưa được đối chiếu đầy đủ; nhánh X2 mới được xác định ở bước gắn callback. Nội dung hai chuỗi mô tả nguồn thưởng và các hàm thống kê trong StateCoin cũng chưa được phân tích hết. Vì vậy, chưa đủ căn cứ để kết luận mọi lần thắng đều tự động cộng ngay 100 coin hoặc có thể can thiệp thành công vào số dư.

8.3 Cách giảm phụ thuộc vào dữ liệu do client quyết định

Với game có máy chủ, có thể giảm phụ thuộc vào client bằng cách lưu số dư và kiểm tra điều kiện nhận thưởng trên máy chủ. Client gửi yêu cầu, còn máy chủ xác định lượng coin hợp lệ và trả về số dư mới. Mỗi lượt thưởng cần có mã riêng để tránh xử lý cùng một yêu cầu nhiều lần. Dữ liệu lưu trên client chỉ dùng để hiển thị tạm và được cập nhật theo kết quả từ máy chủ.

Với game offline, kiểm tra tính toàn vẹn có thể làm việc sửa dữ liệu khó hơn, nhưng dữ liệu vẫn nằm trên thiết bị của người chơi. Cờ _isClaimingReward giúp hạn chế việc nhận thưởng lặp trong luồng giao diện đã xem; cờ này không cho thấy số dư đã được máy chủ xác thực.

9. Kết luận

Qua bài thực hành, đã thực hiện được các bước từ giải nén gói ứng dụng, tìm và xuất asset Unity đến phân tích mã IL2CPP. UABEA được dùng để xem trước và xuất texture; Il2CppDumper tạo dữ liệu tra cứu; dnSpy giúp xác định lớp, phương thức và field; Ghidra được dùng để đọc mã native và lần theo tham chiếu. Kết hợp các công cụ này có thể tìm nơi lưu một giá trị, đối chiếu field offset với lệnh đọc ghi và lần theo luồng xử lý từ giao diện đến phần cập nhật dữ liệu.

Với SandSort, đã xác định được cách nạp số coin khi vào game, giá trị mặc định 100 khi chưa có dữ liệu lưu và cách cộng trừ, lưu số dư. Luồng kiểm tra thắng cũng đã được nối đến thao tác mở UIWinGame và nhận 100 coin qua nút tiếp tục. Kết quả cho thấy cần đọc cả MoveNext của phương thức bất đồng bộ và đối chiếu tên field để hiểu đúng mã giả. Kết quả dựa trên phân tích tĩnh và chưa được đối chiếu bằng việc chạy game.